

UPPSALA
UNIVERSITET

IT mDV 26 035
Degree project 30 credits
August 2026

AI-Assisted Decompilation for Ericsson-Specific Assembly

Jinyi Li



Master's Programme in Computer Science



UPPSALA
UNIVERSITET

AI-Assisted Decompilation for Ericsson-Specific Assembly

Jinyi Li

Abstract

Baseband Firmware Assembly (BFA) is a proprietary Very Long Instruction Word (VLIW)-style assembly language used in Ericsson baseband firmware. BFA routines are handwritten and expose low-level registers, predication, bundle-level parallelism, memory references, and communication primitives, but often lack the source-level structure and symbolic context required for efficient review, debugging, and maintenance. Existing decompilers primarily target standard instruction-set architectures and compiler-generated binaries, making them unsuitable for this proprietary setting where no aligned BFA--C corpus exists.

This thesis investigates whether a hybrid decompilation pipeline can produce faithful and readable C-like pseudo-code under these constraints. The proposed system performs deterministic evidence recovery, including parsing, instruction canonicalisation, control-flow graph construction, intermediate representation lifting, structured control-flow recovery, semantic trigger extraction, and contextual metadata recovery. The recovered evidence is packaged as a structured representation and provided to a large language model (LLM), which acts as an evidence-constrained pseudo-C emitter rather than an end-to-end program analyser.

The pipeline is evaluated on 527 BFA functions using three axes: state consistency, control-flow fidelity, and semantic-token preservation. It achieves mean per-function scores of 0.971, 0.975, and 0.989 respectively. The results show that deterministic evidence recovery effectively anchors LLM-based generation, preserving most operation types, control-flow events, and domain-specific identifiers. Remaining errors mainly arise from ambiguous context, abstraction-related metric artefacts, and rare numeric literal mistakes, suggesting that improving evidence and context recovery is more valuable than relying on larger models alone.

Faculty of Science and Technology

Uppsala University, Place of publication Uppsala

Supervisor: Iuliia Dzevik, Max Östlund Subject reader: Neea Rusch

Examiner: Lars-Henrik Eriksson

Contents

1 Introduction **1**

2 Background **5**

2.1 Input Language: BFA Assembly 5

2.1.1 Execution Model 6

2.1.2 Predicated Execution 6

2.1.3 Communication Primitives 7

2.2 Studies of Decompilation 7

2.2.1 Traditional Decompilation 8

2.2.2 AI-Augmented and Learning-Based Decompilation 9

2.2.3 Positioning of this thesis 10

3 Methodology **13**

3.1 Evidence Recovery Design 13

3.1.1 Syntactic Parsing 13

3.1.2 Instruction Canonicalisation 14

3.1.3 Control-Flow Graph Construction 15

3.1.4 Intermediate Representation Lifting 15

3.1.5 Structured Control-Flow Recovery 16

3.1.6 Semantic Trigger Extraction and Control-Flow Summary . . 17

3.1.7 Context Extraction 17

3.2 Evidence-Constrained Pseudo-C Generation 18

4 Implementation **19**

4.1 Implementation Overview and Scope 19

4.2 Frontend Implementation: Parsing and Canonicalisation 20

4.2.1 Frontend Data Records 20

4.2.2 Bundle-Level Addressing and Fallthrough 21

4.2.3 Operand and Flow-Kind Normalisation 22

4.2.4 Predicate Propagation 23

4.3 CFG and Structural Recovery Implementation 25

4.3.1 Basic Block Construction 25

4.3.2 Typed Edge Construction 26

4.4 IR Lifting Implementation 27

4.4.1 IR Data Model 27

4.4.2 Instruction Translation Dispatch 28

4.4.3 Bundle-Aware IR Emission 28

4.4.4 Predicated Writes 29

4.5 Structured AST Construction 30

4.6 Skeleton Packaging and Evidence Extraction 32

4.6.1 Trigger Record Extraction . . . 32
4.6.2 Control-Flow Summary . . . 33
4.6.3 Context Metadata Recovery . . . 34
4.7 LLM Generation Interface . . . 34
4.7.1 Input Trimming . . . 35
4.7.2 Prompt Template Instantiation . . . 35
4.7.3 Anonymised C-Like Decompilation Output . . . 36

5 Evaluation **39**
5.1 Benchmark Suite . . . 39
5.1.1 Computation and Language-Construct Coverage . . . 40
5.1.2 Representativeness . . . 41
5.2 Axis 1: State Consistency . . . 42
5.2.1 Operation-Type Taxonomy . . . 43
5.2.2 Ground-Truth and Generated-Side Extraction . . . 44
5.2.3 Axis 1 Scoring . . . 44
5.3 Axis 2: Control-Flow Fidelity . . . 45
5.3.1 Structural Event Vocabulary . . . 47
5.3.2 Ground-Truth Event Extraction . . . 47
5.4 Axis 3: Semantic-Token Matching . . . 49
5.4.1 Token Extraction . . . 49
5.4.2 Token Matching and Scoring . . . 51

6 Results **53**
6.1 Overall Results . . . 53
6.2 Per-Axis Detailed Analysis . . . 54
6.2.1 Axis 1: State Consistency . . . 54
6.2.2 Axis 2: CFG Fidelity . . . 56
6.2.3 Axis 3: Semantic Token Matching . . . 57

7 Discussion and Conclusion **61**
7.1 Evaluation Metric Limitations . . . 61
7.2 Context Pipeline Limitations . . . 61
7.3 LLM Capability Boundaries . . . 62
7.4 Error Attribution Summary . . . 62
7.5 Summary and Future Work . . . 63

List of Figures

2.1 General decompilation workflow . . . 8
2.2 Natural loop terminology . . . 9

3.1 BFA decompilation pipeline . . . 14

4.1 Frontend transformation from BFA source to canonical JSON . . . 20
4.2 Bundle-level fallthrough compared with scalar fallthrough . . . 22
4.3 Predicate propagation during canonicalisation . . . 24
4.4 Instruction stream partitioned into basic blocks . . . 26
4.5 Bundle-aware two-phase IR emission . . . 29
4.6 Predicated writes merged through SELECT . . . 30
4.7 Structured AST construction from a typed CFG . . . 31
4.8 Recovering a packed-field access from lifted IR . . . 33

5.1 CFG abstraction gap . . . 46
5.2 Predicated structural operation counting . . . 48
5.3 Shared predicate region in event extraction . . . 48
5.4 Filtering branch instructions for COND events . . . 49

6.1 Pairwise scatter plots of evaluation axes . . . 55
6.2 Axis 1 per-type F1 by instruction-count . . . 55
6.3 Axis 2 fidelity by instruction-count . . . 56
6.4 Axis 3 token recall by instruction-count . . . 58
6.5 Scores versus instruction count . . . 58

List of Tables

2.1 Main lexical categories in BFA source. 5
2.2 Representative decompilation research 11

4.1 Main fields in the frontend JSON records. 21
4.2 Register categories used during operand normalisation. 23
4.3 Control-flow annotation and canonical flow-kind mapping. 23
4.4 Typed CFG edge kinds used during edge construction. 27
4.5 Operand-dependent lowering cases for the `MOV` instruction. 28
4.6 Trigger record categories extracted from the lifted IR. 34
4.7 Sanitised prompt-template layers for evidence-constrained pseudo-C generation. 36

5.1 Benchmark-suite coverage by construct 41
5.2 Benchmark-suite coverage by module category 42
5.3 Sanitised operation-type taxonomy for state-consistency evaluation. 43

6.1 Mean evaluation scores by function instruction-count 54
6.2 Distribution of per-function evaluation scores 54
6.3 Axis 1 categories with systematically lower F1 scores 54
6.4 Axis 2 mean score by instruction-count bin. 56

7.1 Attribution of score reductions across all 527 BFA functions. Files may appear in multiple rows where issues co-occur. 63

Chapter 1

Introduction

Decompilation aims to recover a human-readable high-level representation from low-level code, and it remains an important capability in reverse engineering, software maintenance, debugging, and program comprehension. The task is inherently difficult because low-level code often lacks source-level abstractions such as variable names, type information, comments, and structured control flow. Cifuentes [1994] established decompilation as a program recovery problem rather than a simple syntactic transformation. More recent work shows that modern decompilers still face limitations in correctness, fidelity, and readability, while neural and hybrid approaches introduce new opportunities as well as new risks related to faithfulness and transferability [Dramko et al., 2024; Liu and Wang, 2020; Tan et al., 2024]. These observations motivate staged decompilation pipelines in which deterministic program analysis recovers low-level evidence, while learned generation is used only after that evidence has been made explicit.

This thesis addresses decompilation in a proprietary industrial setting centred on *Baseband Firmware Assembly* (BFA), an Ericsson-specific assembly-level source language used in baseband firmware. Unlike many standard decompilation targets, the BFA routines studied in this work are handwritten by developers rather than generated automatically by a compiler. Consequently, there is no naturally aligned corpus pairing BFA routines with equivalent implementations in the C programming language for direct supervision, no existing decompiler output that can be reused as an intermediate representation, and only limited applicability of general-purpose decompilation tools. The problem therefore differs from settings that assume mainstream instruction-set architectures, compiler-generated binaries, and publicly available paired corpora.

The practical motivation for this work is program comprehension. BFA routines are low-level artefacts that may contain explicit register operations, predicated instructions, bundle-level parallelism, control-transfer operations, memory references, and communication events. Although such programs are executable and precise, they are difficult to inspect manually because the developer-intended structure is not always explicit in the source representation. For engineers who need to understand, review, debug, or maintain these routines, a faithful C-like pseudo-code representation can provide a more accessible view of the same behaviour. However, producing such a representation requires more than replacing instruction mnemonics with C-like syntax. The output must improve readability while still preserving control-flow structure, side effects, data dependencies, predicates, and architecture-specific execution rules.

Problem Formulation

The problem addressed in this thesis is not general binary decompilation, but faithful pseudo-C recovery for handwritten BFA source routines under industrial constraints. The input is a textual low-level routine containing explicit register operations, predication, bundle-level parallelism, memory references, control transfers, and communication events. The desired output is a readable C-like pseudo-code representation that supports program comprehension while preserving the behaviourally important evidence present in the input.

This problem has three concrete challenges. First, the source language is proprietary and does not match the assumptions of mainstream decompilers for standard instruction-set architectures. Second, the routines are handwritten, so there is no compiler-generated structure that can simply be inverted and no naturally aligned BFA–C-language corpus for supervised translation. Third, important semantics are distributed across local instructions, control-flow structure, and surrounding repository context. The study therefore asks whether deterministic evidence recovery can make these facts explicit enough for a large language model (LLM) to render them into readable pseudo-C without becoming an unconstrained translator.

The goal of this thesis is therefore to build and evaluate a hybrid pipeline for decompiling BFA into C-like pseudo-code. The proposed approach first applies deterministic evidence recovery to make control flow, data flow, instruction effects, and contextual metadata explicit. This recovered evidence is packaged into a structured evidence representation, later referred to as a skeleton, and used to constrain LLM-based pseudo-C generation. The model is therefore not asked to infer program behaviour directly from raw assembly, but to render a readable output grounded in deterministic analysis results. In terms of a traditional decompilation workflow, the LLM replaces the final rule-based pseudo-C generation backend, while the frontend and middle-end recovery tasks remain deterministic.

In this thesis, *fidelity* is defined with respect to the evidence recoverable from the BFA input, rather than full semantic equivalence with an unavailable original C implementation. A generated pseudo-C output is considered faithful when it preserves the behaviourally important facts recovered by the deterministic pipeline: operation-level effects, control-flow event structure, and domain-specific semantic tokens. This definition intentionally excludes properties that cannot be reliably recovered from the available input, such as original source-level variable names, precise C types, comments, and fully compilable C syntax. Readability is treated as a design objective of the pseudo-C rendering, while the quantitative evaluation focuses on fidelity to the recovered evidence.

Research Scope

This thesis is an artefact-building and empirical evaluation study. It produces a working evidence-constrained decompilation pipeline and evaluates the fidelity of its generated pseudo-C outputs on a dataset of BFA functions. The work focuses on textual BFA source artefacts rather than native binary decoding. Within this scope, the deterministic stages recover program evidence from the input routine, including instruction effects, control-flow structure, data-flow evidence, semantic triggers, and selected contextual metadata. The LLM stage then renders this evidence into a readable C-like form.

The research also includes a task-specific evaluation framework. Since the generated output is pseudo-C rather than necessarily compilable C, the evaluation does not rely on exact textual similarity. Instead, it separates faithfulness into three measurable dimensions: operation-level consistency, control-flow event preservation, and semantic-token preservation. These axes assess whether the generated pseudo-C preserves the behaviourally important evidence recovered from the BFA input, even when the surface syntax differs from the original low-level representation.

Delimitations

This thesis does not aim to build a general-purpose decompiler for all architectures or instruction sets. The pipeline is specialised to the BFA setting and to the forms of evidence available in the studied industrial codebase. It also does not attempt to recover fully compilable C source code. The intended output is readable pseudo-C for program comprehension, not a drop-in replacement for the original low-level program.

The thesis does not attempt to replace deterministic program analysis with an end-to-end LLM. Parsing, canonicalisation, control-flow graph construction, intermediate-representation lifting, and structured control-flow recovery remain deterministic. The LLM is introduced only after these stages, where it acts as an evidence-constrained pseudo-C emitter. This design is intentional: low-level program facts should be recovered mechanically where possible, while the model is used for readability-oriented rendering, naming, expression selection, and explanation.

The thesis also does not focus on training or fine-tuning a new model. Instead, it investigates whether an existing LLM can be used effectively when supplied with a carefully constructed evidence skeleton. Deployment concerns, such as a production IDE plugin or continuous integration with internal tooling, are outside the main research scope. Finally, although readability motivates the choice of pseudo-C as the output form, this thesis does not conduct a separate human-subject readability study; the empirical evaluation focuses on measurable fidelity to recovered program evidence.

Research Questions

Based on these constraints and goals, the overarching research question of this thesis is:

> **RQ:** To what extent can an evidence-constrained hybrid decompilation pipeline, combining deterministic BFA evidence recovery with LLM-based pseudo-C generation, preserve the behaviourally important evidence of handwritten BFA routines, and what factors limit this fidelity?

This overarching question is operationalised through four more specific research questions:

> **RQ1:** To what extent does the generated pseudo-C preserve the instruction-level operation categories recovered from the BFA input?
>
> **RQ2:** To what extent does the generated pseudo-C preserve the control-flow event skeleton of the original BFA routine, including conditional

branches, switch-like dispatches, and externally visible communication events?

RQ3: To what extent does the generated pseudo-C preserve concrete semantic tokens recovered from the BFA evidence, such as constants, register-derived values, offsets, symbolic operands, and field-like memory references?

RQ4: When fidelity scores are reduced, are the losses primarily caused by evaluation artefacts, incomplete input context, or limitations of the LLM generation stage?

RQ1 is evaluated using the state-consistency metric, RQ2 using the control-flow-fidelity metric, RQ3 using semantic-token matching, and RQ4 through manual error attribution over the observed score reductions. This formulation makes the research questions measurable while preserving the broader goal of studying BFA decompilation as a program-comprehension problem.

Contributions

This thesis makes four main contributions:

1. It presents an end-to-end hybrid decompilation pipeline for BFA, combining deterministic evidence recovery with LLM-based pseudo-C generation.
2. It implements a deterministic evidence layer that recovers canonical instructions, CFG structure, lifted IR, structured control-flow evidence, semantic triggers, and selected repository context.
3. It introduces a context-extraction component that recovers data-layout definitions, caller metadata, execution-point information, and communication-related metadata when these can be determined statically.
4. It proposes and applies a three-axis evaluation framework for pseudo-C fidelity; the framework measures operation consistency, control-flow event preservation, and semantic-token preservation across 527 BFA functions.

Chapter 2

Background

2.1 Input Language: BFA Assembly

BFA is the assembly-level input language studied in this thesis. A BFA routine is a textual low-level source artefact consisting of labels, instruction bundles, operands, predicates, memory references, and communication primitives. The input to the decompiler is therefore not a native binary, but a handwritten source-level routine intended to control specialised firmware behaviour.

The examples in this chapter are schematic and sanitised. They are included to show the kinds of language constructs handled by the pipeline, without exposing proprietary instruction names or internal data layouts. A simplified BFA-like routine may contain labelled control-flow targets, register updates, memory accesses, and message dispatches, for example:

```
entry:
    LD    r0, [ctx + #state.counter]
    ADD   r0, r0, #1
    ST    r0, [ctx + #state.counter]
    SEND  donePort, r0
```

In this example, `entry` is a label, `r0` is a register-like state location, `state.counter` is a symbolic memory field, and `donePort` is a communication endpoint. The role of the decompiler is not only to translate individual mnemonics, but also to preserve the control flow, side effects, and semantic identifiers that make the routine understandable.

The simplified example also illustrates the main lexical categories that appear in BFA source. Table 2.1 summarises these categories at the language level. The implementation chapter later explains how the parser recognises these forms and converts them into canonical records.

Table 2.1: Main lexical categories in BFA source.

Category	Examples	Role
Mnemonic	`mov`, `add`, `cmp`, `jmp`	Identifies the operation of an instruction.
Register	`r0`, `r1`, `sp`, `pc`	Names architectural state used by instructions.

Continued on next page

Table 2.1: Main lexical categories in BFA source (continued).

Category	Examples	Role
Immediate	`#12`, `0x10`, `SIZE`	Represents literal or symbolic constants.
Memory reference	`[r1 + 4]`, `4(r1)`	Represents indirect access through a base register and displacement.
Label	`loop:`, `target:`	Marks symbolic branch targets or source positions.
Condition / predicate	`eq`, `ne`, `lt`	Represents branch conditions or instruction guards.
Bundle marker	`||`, bundle boundary	Groups operations belonging to the same VLIW issue bundle.
Structural punctuation	`,`, `[ ]`, `( )`	Separates operands and delimits addressing expressions.

2.1.1 Execution Model

BFA follows a Very Long Instruction Word (VLIW) execution model [Hennessy and Patterson, 2019]. In a VLIW architecture, one instruction bundle may encode several operations that are intended to issue in the same cycle or logical execution step. Instead of relying on dynamic hardware scheduling to discover parallelism at run time, the compiler or programmer is responsible for grouping independent operations statically.

As in architectures such as the TI TMS320C6x family [Texas Instruments, 2016] or the Itanium EPIC architecture [Intel Corporation, 2002], the hardware performs no dynamic scheduling, register renaming, or out-of-order execution; all parallelism is determined at compile time. In the textual representation, parallel operations within a bundle are delimited by a pipe symbol (`|`), analogous to the semicolons used in TI C6x assembly to mark parallel issue groups.

In this thesis, BFA execution is treated as bundle-level parallel rather than purely sequential. The textual order of operations inside one bundle is not interpreted as a sequence of reads and writes visible to later operations in the same bundle. Instead, all operations in the bundle observe the state at bundle entry, and their writes become visible only after the bundle has been evaluated. When the implementation later emits a sequential intermediate representation, it therefore has to model this behaviour explicitly rather than lowering bundle contents as an ordinary instruction list.

2.1.2 Predicated Execution

BFA supports predicated, or guarded, execution in a manner similar to ARM's conditional execution field or Itanium's qualifying predicates [ARM Limited, 2012; Intel Corporation, 2002]. A predicate is a boolean guard that determines whether an operation commits its result. Conditional behaviour therefore appears in more than one form:

- **Control-flow branching**: a conditional branch that creates divergent paths in the control-flow graph (CFG).
- **Instruction-level predication**: a per-instruction guard analogous to Itanium's qualifying predicate register. The guarded instruction conditionally commits its result while the program counter advances normally.

– **Bundle-level conditional grouping**: an `IF/ELSE` construct that partitions instructions within a single bundle into two mutually exclusive groups, both residing at the same program address.

The distinction is important for decompilation. A branch changes the CFG and may become an `if`, loop, or switch-like construct in pseudo-C. A predicated non-branch instruction, by contrast, does not introduce a new CFG edge by itself; it is better represented as a guarded assignment or conditional expression. The decompiler must therefore distinguish branch-based control flow from predicated data-flow updates.

2.1.3 Communication Primitives

Unlike general-purpose VLIW processors optimised mainly for arithmetic throughput, BFA includes explicit send-like instructions that dispatch messages to external hardware subsystems—comparable to the channel operations found in network processor architectures such as Intel IXP [Intel Corporation, 2003]. These operations constitute observable side effects and must be faithfully preserved in the decompiled output. In the generated pseudo-C, the exact low-level send encoding may be abstracted, but the existence of the communication event and its destination endpoint remain behaviourally important evidence.

2.2 Studies of Decompilation

Following standard accounts of reverse compilation, decompilation is understood as the process of recovering a high-level, source-code-like representation from a low-level program representation, such as executable code, machine code, or assembly-level code [Cifuentes, 1994; Cifuentes and Gough, 1995]. In native-code decompilation, the recovered program is frequently rendered as C-like pseudo-code, although other decompilation settings may target other source languages. This thesis follows the C-like pseudo-code convention because the goal is program comprehension rather than recompilation.

Decompilation is not merely a textual translation task. The goal is instead to construct a representation that is easier for humans and program-analysis tools to understand while remaining faithful to the behaviour of the low-level input. This requires the recovery of abstractions that are either absent or only implicitly represented at the low level. Variable names, comments, source-level types, structured control constructs, and developer-intended program structure may be partially unavailable, completely removed, or recoverable only through inference. In compiler-generated binaries, this difficulty is amplified by instruction selection and optimisation, which may replace source-level structure with lower-level control transfers and data manipulations that are not straightforward to invert. More generally, decompilation is therefore best viewed as a problem of behaviour-preserving abstraction recovery rather than as a formatting problem.

Early work by Cifuentes [1994] and Cifuentes and Gough [1995] provided foundational systematic treatments of reverse compilation, framing the decompiler as a multi-stage system involving machine-dependent front-end analysis, intermediate program representation, control-flow and data-flow recovery, type reasoning, and source-level code generation. Later work further emphasised that high-quality decompilation depends not only on producing readable C-like syntax, but also on recovering abstractions correctly, especially structured control flow and faithful program behaviour [Basque et al., 2024; Schwartz et al., 2013; Yakdan et al., 2015].

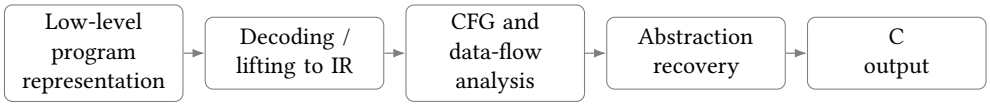


Figure 2.1: A simplified general decompilation workflow. The pipeline first translates low-level program evidence into an intermediate representation (IR), then applies control-flow and data-flow analyses to recover higher-level abstractions before rendering a pseudo-C output.

2.2.1 Traditional Decompilation

Classical decompilation pipelines typically involve several interdependent stages, including lifting low-level code into an intermediate representation (IR), recovering the control-flow graph (CFG), performing data-flow and memory analysis, inferring types, and structuring the recovered graph into conditionals, loops, and other source-like constructs. Figure 2.1 summarises this general workflow at a high level. The figure is intentionally simplified: real decompilers often iterate between these stages, but the diagram captures the main progression from low-level evidence recovery to source-like rendering.

Control-flow structuring concepts. Control-flow structuring commonly relies on graph relations such as dominance, post-dominance, and natural-loop detection. These concepts are standard in compiler control-flow analysis [Aho et al., 2007; Lengauer and Tarjan, 1979] and are widely used in decompilation to recover source-level constructs from low-level control-flow graphs [Cifuentes, 1994; Schwartz et al., 2013].

A block A *dominates* a block B if every path from the entry block to B passes through A. Dominance is useful for loop recovery. In a natural loop, a back edge from a latch block L to a header block H is identified when the target dominates the source:

$$(L, H) \text{ is a back edge} \iff H \in \mathrm{dom}(L).$$

The target H is the loop header, and the source L is a latch block. The loop body consists of the header together with the blocks that can reach the latch without passing through the header. Figure 2.2 illustrates these terms for a simple natural-loop pattern.

Post-dominance provides the corresponding concept for branch reconvergence. A block A *post-dominates* a block B if every path from B to the program exit passes through A. For a branch block B_0, the immediate post-dominator identifies the merge point where the branch arms reconverge. This boundary is important during decompilation because it prevents code after a branch from being incorrectly absorbed into either the true or false branch body.

With these graph concepts in place, control-flow structuring can be understood as the decompiler stage that rewrites low-level CFG regions into source-level constructs. Among the stages in Figure 2.1, control-flow structuring has been one of the most intensively studied because it strongly affects both semantic faithfulness and readability.Schwartz et al. [2013] emphasise semantics-preserving structural analysis and iterative control-flow structuring as a way to recover source-level structure without changing program meaning. Yakdan et al. [2015] further argue that high-quality structuring should not depend purely on predefined control-flow patterns

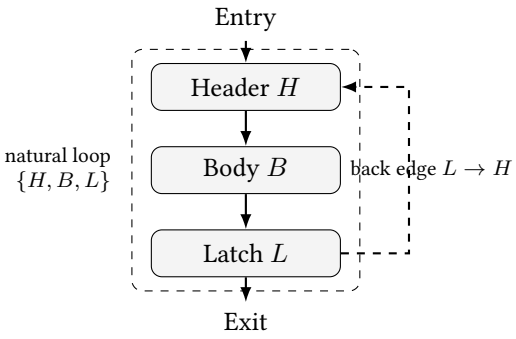


Figure 2.2: Basic terminology for dominance-based natural loop detection. A back edge from a latch block L to a header block H identifies a natural loop when H dominates L.

and propose pattern-independent, semantics-preserving transformations to reduce unnecessary `goto` statements in decompiled output. Taken together, this line of work shows that successful decompilation depends not only on decoding instructions correctly, but also on reconstructing higher-level program structure in a way that remains faithful to the original semantics.

A second important strand of traditional decompilation research concerns evaluation. Existing work shows that decompiler quality cannot be assessed only by whether the output appears syntactically plausible. Correctness-oriented evaluations use recompilation and execution-based analysis to test whether decompiled code preserves the behaviour of the original program, revealing that readable-looking output may still be semantically incorrect [Liu and Wang, 2020]. Other studies broaden the evaluation target from functional correctness to practical effectiveness, including semantic consistency, structural fidelity, and readability [Burk et al., 2022; Cao et al., 2024; Dramko et al., 2024; Eom et al., 2024]. These studies are relevant to this thesis because they motivate evaluating decompilation quality along multiple dimensions rather than relying on surface similarity alone.

This thesis builds on three established ideas from traditional decompilation. First, it follows the frontend/middle-end/backend separation in which target-specific input is normalised before source-like code is emitted [Cifuentes, 1994; Cifuentes and Gough, 1995]. Second, it uses the standard compiler representation of programs as basic blocks connected by a control-flow graph, which provides the basis for branch and loop recovery [Aho et al., 2007; Allen, 1970]. Third, it follows semantics-preserving control-flow structuring work, where low-level jumps and edges are reorganised into source-level constructs such as conditionals and loops without changing program behaviour [Schwartz et al., 2013; Yakdan et al., 2015].

Positioning. The novelty of this thesis is not the use of these classical analyses in isolation, but their use as an evidence-recovery front end for an LLM-based pseudo-C emitter in a proprietary handwritten assembly setting.

2.2.2 AI-Augmented and Learning-Based Decompilation

Alongside classical approaches, learning-based decompilation has emerged as an increasingly important research direction. One major line of work treats decompilation as a translation problem. Katz et al. [2019] frame decompilation as a neural machine

translation task and show that low-level representations can be translated back into source-like code when training data provides aligned low-level and source-level examples, usually for standard architectures and compiler-generated programs. Hosseini and Dolan-Gavitt [2022] extend this line of thinking by proposing a retargetable formulation that reduces language-specific engineering, but still treats decompilation primarily as sequence translation. Cao et al. [2022] further show that neural methods can improve decompilation quality for optimised binaries, highlighting the ability of data-driven methods to recover source-like structure from challenging low-level inputs.

These translation-based approaches demonstrate the potential of learned decompilation, but they also illustrate a restriction that is important for this thesis. Their success is closely tied to settings in which suitable training pairs, benchmark corpora, or compiler-generated examples are available. This is different from the BFA setting studied here, where the input is a proprietary handwritten assembly-level source language and no naturally aligned BFA–C programming language corpus is available for supervised translation.

Another line of work uses AI not as a replacement for decompilation, but as a post-processing or enrichment stage after deterministic recovery has already taken place. Chen et al. [2022], for example, observe that decompilers can reconstruct many structural properties of code, but that source-level information such as meaningful variable names and custom types is typically lost during compilation. Their DIRTY system therefore applies a learned model to existing decompiler output in order to recommend variable names and types from the surrounding code context.

This staged use of AI is closer to the present thesis than end-to-end sequence translation, but the setting and role of the learned model are different. DIRTY assumes that a conventional decompiler has already produced a C-like output, whereas no such BFA decompiler output is available here. DIRTY enriches an existing decompilation mainly with names and types, while this thesis first constructs a deterministic evidence layer from the BFA input itself and then uses an LLM as the final pseudo-C emitter. The model is therefore not merely renaming variables in an existing decompilation; it is rendering a structured evidence package into readable pseudo-C under explicit faithfulness constraints.

At the same time, advances in learning-based decompilation have been closely tied to the availability of larger datasets and stronger models. ExeBench provides a large benchmark corpus of executable C functions for program-recovery research [Armengol-Estapé et al., 2022]. SLaDe studies portable decompilation across architectures and optimisation levels, while LLM4Decompile illustrates the growing interest in end-to-end large-language-model-based decompilation and refinement on standard benchmarks [Armengol-Estapé et al., 2024; Tan et al., 2024]. These works are not used as datasets or baselines in this thesis, as due to confidentiality issue no external decompiler tool is allowed to work on Ericsson BFA product code. They are included to characterise the research context: recent learning-based decompilation has progressed mainly in data-rich, standardised settings, whereas the present work studies a proprietary industrial setting where such data is unavailable.

2.2.3 Positioning of this thesis

Most existing decompilation literature—classical or learning-based—assumes standard instruction-set architectures, compiler-generated binaries or assembly, and, in the learning-based case, some form of aligned source-level supervision. These as-

sumptions do not hold in the BFA setting.

The handwritten nature of BFA is important because the input is not the result of compiling a known high-level source program. There are therefore no compiler idioms, optimisation patterns, or aligned source snippets that can be treated as the inverse target of decompilation. Program structure is instead expressed through low-level control transfers, predicates, symbolic operands, and domain-specific communication conventions. In addition, some information needed for readable pseudo-C generation is not contained in the local routine alone. For example, the meaning of a memory displacement may be defined in a separate layout declaration, the destination of a communication primitive may be documented in interface metadata, and the correct interpretation of a routine may depend on the caller or execution-entry point.

Table 2.2 summarises representative decompilation research by paradigm and positions this thesis within the existing literature.

Table 2.2: Representative decompilation research and how each paradigm informs the positioning of this thesis.

Research paradigm	**Representative work**	**Connection to this thesis**
Classical decompilation and structuring	[Cifuentes, 1994; Schwartz et al., 2013; Yakdan et al., 2015]	Provides the foundation for deterministic recovery of control flow, intermediate evidence, and source-like structure before generation.
Decompiler evaluation	[Burk et al., 2022; Cao et al., 2024; Dramko et al., 2024; Eom et al., 2024; Liu and Wang, 2020]	Motivates evaluating fidelity along multiple dimensions rather than relying on surface similarity alone.
Neural decompilation	[Cao et al., 2022; Hosseini and Dolan-Gavitt, 2022; Katz et al., 2019]	Shows the potential of learned translation, but typically assumes aligned training data, compiler-generated examples, or standard architectures.
Hybrid AI-assisted decompilation	[Chen et al., 2022]	Motivates using learned models after deterministic recovery, but differs from this thesis because it enriches existing decompiler output rather than building the evidence layer for a proprietary input language.
Large-scale benchmarks and LLM-based methods	[Armengol-Estapé et al., 2022, 2024; Tan et al., 2024]	Provides research context for modern data-driven decompilation, but under more standardised and data-rich conditions than the BFA setting.

Taken together, the literature suggests that neither a purely classical nor a purely end-to-end neural formulation can be transferred directly to the present setting. Classical decompilation motivates the deterministic recovery of control flow, data flow, and structured program evidence, but it does not by itself provide a ready-made backend for this proprietary source language. End-to-end neural decompilation motivates the use of learned generation, but it typically depends on aligned corpora and standardised benchmarks that are not available for BFA.

The expected benefit of combining the two approaches is therefore not that their limitations disappear, but that they are assigned to different parts of the pipeline. Deterministic analysis is used for facts that must be mechanically preserved, such as instruction effects, control-flow edges, predicates, and side effects. The LLM is used

only for the less rigid rendering task: producing readable pseudo-C, selecting expressions, introducing names when supported by evidence, and explaining unresolved cases. Limitations may still arise when the deterministic evidence is incomplete or when the model misuses ambiguous context; these limitations are therefore explicitly evaluated later in the thesis.

Chapter 3

Methodology

This chapter describes the methodological design of the BFA decompilation pipeline. The design separates deterministic evidence recovery from high-level pseudo-C generation. Rather than asking a large language model (LLM) to translate raw BFA assembly directly into source-like output, the pipeline first transforms BFA source artefacts into a structured, machine-readable JSON evidence package. This thesis uses the term *skeleton* for this package. The skeleton contains the recovered program evidence that later stages may use, including normalised instructions, control-flow information, lifted intermediate representation (IR), semantic triggers, and contextual metadata. Figure 3.1 provides an overview of the proposed BFA decompilation pipeline.

The generation stage does not assume that the LLM is faithful by construction. Instead, the methodology restricts the model's role: the deterministic skeleton remains the authoritative source of program facts, and the prompt and output schema are designed to keep the generated pseudo-C grounded in that evidence. The extent to which this constraint is satisfied is evaluated empirically in Chapter 5.

3.1 Evidence Recovery Design

The evidence recovery design separates deterministic program analysis from generative code production. The pipeline first recovers a stable evidence layer from source artefacts, including normalised instructions, control-flow structure, lifted intermediate semantics, semantic triggers, and contextual metadata. This evidence layer is then passed to the LLM as an explicit grounding source, rather than allowing the model to infer program behaviour directly from raw assembly text.

3.1.1 Syntactic Parsing

The first step in structural recovery is to convert the preprocessed BFA source text into an initial parser-level abstract syntax tree (AST) that reflects the surface structure of the source program. This follows the conventional compiler frontend separation between lexical analysis and parsing: the lexer first tokenises the input character stream, and the parser then organises the token stream according to the grammar of the input language Aho et al. [2007]. In this work, however, the input is not a high-level programming language and not a native binary. It is a textual, assembly-like BFA source artefact whose syntax is organised around instruction bundles, predicates, labels, and target-specific operands. The parser therefore performs source-level syntactic recovery rather than binary instruction decoding.

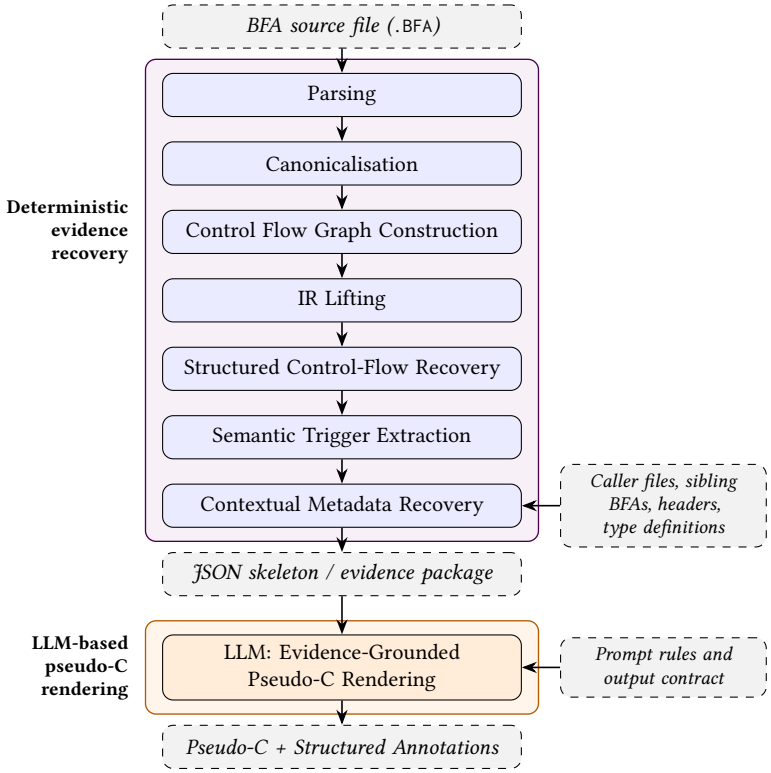


Figure 3.1: Overview of the BFA decompilation pipeline. The stages labelled *Deterministic evidence recovery* transform a BFA source file into a structured JSON skeleton. The stage labelled *LLM-based pseudo-C rendering* uses this skeleton, together with prompt rules and an output contract, to produce pseudo-C and structured annotations. Dashed boxes denote auxiliary inputs or intermediate/output artefacts.

Lexical structure. BFA source follows a VLIW-style bundle structure. Each bundle contains one or more instructions, and each instruction consists of a mnemonic followed by zero or more operands. The lexer recognises instruction mnemonics, operands, labels, predicates, and structural punctuation.

Parser Design. The parser consumes the token stream bundle by bundle rather than instruction by instruction. This design follows the VLIW-style organisation of BFA source, where multiple operations may belong to the same issue unit instead of forming a purely sequential instruction stream Hennessy and Patterson [2019]. The parser therefore groups all operations in the same bundle under a single AST bundle node, preserving the source-level parallel structure for later analysis. This bundle node later becomes the atomic unit for address assignment and CFG partitioning: instructions inside the same bundle are analysed as separate operations, but they are not split across different basic blocks.

3.1.2 Instruction Canonicalisation

After syntactic parsing, the parser-level AST is transformed into a canonical instruction representation. The purpose of this step is to separate source-specific syntax from the semantic evidence required for later decompilation stages. This follows a common pattern in decompiler frontends: instruction forms are normalised into a

representation that is more suitable for control-flow recovery, data-flow analysis, and semantic lifting than the original surface syntax Cifuentes [1994]; Kim et al. [2017]. This stage does not attempt to recover high-level program structure. It does not introduce loops, conditionals, or pseudo-C constructs. The output is still a low-level representation, but it is regular enough for the CFG builder and IR lifter to operate without repeatedly reasoning about parser artefacts or target-specific surface syntax.

3.1.3 Control-Flow Graph Construction

The canonical instruction representation produced by the previous stage is converted into a typed control-flow graph (CFG). This stage follows a standard compiler and decompiler frontend pattern: a low-level program representation is partitioned into basic blocks, which are then connected by explicit control-flow edges Aho et al. [2007]; Cifuentes [1994]; National Security Agency [2026].

In this work, however, CFG construction is performed at bundle granularity rather than over a purely scalar instruction stream. Because BFA is organised around VLIW bundles, all instructions in the same bundle share one synthetic address and remain in the same basic block. A basic block is therefore a maximal contiguous range of bundles with a single block-level entry and a single block-level exit. Block boundaries may occur before the first bundle of the routine, before a resolved branch or dispatch target, or before the fallthrough successor of an explicit control-flow transfer. They do not occur between sibling instructions inside the same bundle, since that would incorrectly turn parallel bundle operations into a sequential instruction chain.

The CFG is constructed statically from the canonical BFA representation, where labels, fallthrough information, branch annotations, and switch metadata have already been made explicit by the preceding stages. Labels resolve to bundle addresses, and fallthrough successors refer to the next bundle rather than to the next instruction within the same bundle. Edges then represent possible transfers of control between basic blocks and are annotated with their control-flow role, such as fallthrough, conditional branch, unconditional jump, switch case, default dispatch, or exit.

The resulting typed CFG provides the control-flow backbone for the rest of the pipeline. Intermediate-representation lifting attaches block-local semantic operations to the CFG nodes, while structured control-flow recovery uses the typed edges and graph topology to recover higher-level constructs such as conditionals, loops, and switches.

3.1.4 Intermediate Representation Lifting

After control-flow recovery, each basic block is lifted from its contained BFA bundles and canonical instruction records into a semantic intermediate representation (IR). IR lifting is a standard compiler and decompiler design pattern: target-specific instructions are translated into an intermediate form before subsequent analysis, optimisation, or source-level reconstruction is performed Cifuentes [1994]; Lattner and Adve [2004]; Schwartz et al. [2013]. In this thesis, the IR is not proposed as a new general-purpose IR. Its role is to make the operational evidence recovered from BFA explicit and machine readable before pseudo-C generation.

The design follows standard static-single-assignment-style (SSA-style) data-flow representation Cytron et al. [1991], but adapts it to the BFA setting in three ways:

(a) **CFG preservation.** IR statements are attached to basic blocks, while the inter-block CFG topology, edge types, and terminators recovered in the previous stage are preserved. The CFG records where execution may go; the IR records what each block does.

(b) **Explicit architectural state.** Register-like reads, writes, memory accesses, and side effects are represented explicitly rather than left implicit in instruction syntax. Temporary SSA-style values make data dependencies visible to later stages.

(c) **BFA-specific execution semantics.** Bundle-level parallelism, predicated non-branch instructions, and communication operations are represented conservatively. Bundle contents are not treated as an ordinary sequential instruction list; predicated data updates are not automatically converted into CFG branches; and communication operations are preserved as observable side effects.

This IR-CFG therefore separates control-flow evidence from data-flow evidence: control-flow recovery determines the possible execution paths, while IR lifting records the block-local instruction effects in a form that can be consumed by trigger extraction, structuring, and LLM-based pseudo-C generation.

3.1.5 Structured Control-Flow Recovery

The final structural recovery stage transforms the recovered typed CFG into a hierarchical structured AST. This AST is distinct from the parser-level AST introduced in Section 3.1.1. The parser-level AST is a flat, source-order representation of bundles, labels, operands, and macro calls; it preserves the surface syntax of the BFA source but does not encode recovered control-flow structure. In contrast, the structured AST represents recovered source-level constructs such as sequences, conditionals, loops, switches, exits, breaks, and continuations.

This stage applies the control-flow structuring concepts introduced in Section 2.2.1. The structurer operates over the typed-edge CFG produced by the previous stages. Typed edges identify the semantic role of each control transfer, such as fallthrough, conditional branch, unconditional jump, switch case, default dispatch, or exit. Post-dominance is used to identify reconvergence points for acyclic conditional regions, while dominance-based back-edge detection is used to identify natural loop regions. The purpose is not to invent new control-flow semantics, but to reorganise the recovered low-level CFG into a source-like hierarchy when the graph evidence supports doing so.

The methodological choice in this thesis is to perform structuring before LLM-based generation. The LLM therefore receives explicit structured control-flow evidence instead of having to infer conditionals, loops, and switches directly from raw assembly or from an unstructured graph. When a region can be matched safely, the structurer emits the corresponding high-level node, such as `IfElse`, `While`, or `Switch`. When the evidence is ambiguous or the region cannot be structured without changing the recovered control flow, the pipeline preserves the transfer explicitly using continuation or fallback nodes rather than forcing an incorrect high-level construct.

The resulting structured AST is passed forward as high-level control-flow evidence for pseudo-C generation. Together with the typed CFG, it allows later stages to distinguish between low-level graph evidence and recovered source-level structure: the CFG remains available as the exact control-flow record, while the structured AST provides the preferred representation for readable pseudo-C output.

3.1.6 Semantic Trigger Extraction and Control-Flow Summary

The IR-CFG and structured AST produced by the preceding stages contain the main semantic evidence for pseudo-C generation, but this evidence is distributed across fine-grained SSA statements or represented only implicitly in graph topology. Before generation, the pipeline therefore performs a deterministic evidence-packaging step that converts selected IR and CFG facts into flat, queryable records.

The trigger extraction pass summarises recurring semantic patterns from the IR-CFG, such as memory references, packed-field accesses, symbolic constants, communication operations, and entry or continuation markers when they are recoverable from the deterministic evidence. Its purpose is not to infer new semantics, but to expose already recovered low-level evidence in a form that the generation stage can use directly. In parallel, selected control-flow information is serialised as a flat summary of typed edges, branch conditions, dispatch targets, and block terminators. This summary complements the structured AST: the AST provides the preferred hierarchical representation, while the flat CFG summary preserves exact low-level topology when a region is only partially structured or requires a fallback representation.

The output of this stage is a set of skeleton-derived evidence records attached to the JSON skeleton. JSON is used as the interface format because it provides a lightweight, text-based, language-independent representation of structured data [Bray, 2017]. This is suitable for the skeleton because the recovered evidence is naturally hierarchical: instructions belong to bundles and basic blocks, basic blocks are connected by typed CFG edges, and trigger records refer back to IR statements or canonical instructions. A JSON representation also keeps the evidence machine-readable, so later stages can validate required fields, inspect provenance links, and selectively trim or serialise the evidence before generation.

JSON is also a practical interface for LLM-based generation. Modern LLM tool-use and structured-output interfaces commonly describe inputs and outputs using JSON objects or JSON Schema-like specifications. In this thesis, JSON is not used because it guarantees model faithfulness, but because it provides a structured and inspectable carrier for deterministic evidence. These records reduce the amount of symbolic reconstruction expected from the LLM and help keep pseudo-C generation grounded in deterministic analysis results.

3.1.7 Context Extraction

The deterministic evidence recovered from a single BFA source file is not always sufficient for faithful pseudo-C generation. Many low-level instructions refer to information that is defined outside the local instruction stream, such as caller-side invocation logic, symbolic constants, memory layout declarations, packed data formats, communication endpoints, and resume-entry metadata. Therefore, the pipeline includes a repository-aware context extraction stage before the LLM-based generation step Hennessy and Patterson [2019].

This stage takes two inputs: the path of the BFA artefact being decompiled and the skeleton JSON produced by the preceding recovery stages. It returns a context dictionary that is attached to the skeleton as additional grounding evidence. The extraction is deterministic and rule-based: rather than asking the LLM to infer missing background information from assembly text, the system uses hand-written static lookup rules to recover relevant context from surrounding repository artefacts.

The output of this stage is not intended to be a complete model of the whole codebase. Instead, it is a compact evidence package containing the contextual facts

most relevant to decompilation. When a piece of context cannot be resolved statically, it is left unresolved rather than inferred. This preserves the boundary between deterministic evidence recovery and generative reconstruction: the LLM may use the recovered context to produce more meaningful pseudo-C names, comments, and control-flow explanations, but it is not expected to invent repository-specific semantics that are absent from the evidence layer.

3.2 Evidence-Constrained Pseudo-C Generation

After deterministic evidence recovery, the pipeline has produced a structured skeleton containing the program evidence needed for generation. At this point, the goal is not to produce compilable or executable C code. The generated output is C-like pseudo-C intended for program comprehension, inspection, and debugging. The remaining task is therefore not to rediscover program structure from raw assembly, but to render the recovered evidence in a human-readable form.

In terms of the traditional decompilation workflow, the LLM is introduced at the final source-level rendering stage. It does not replace the frontend or middle-end analysis stages of a decompiler, such as parsing, canonicalisation, CFG construction, IR lifting, or structured control-flow recovery. Instead, it replaces a purely rule-based pseudo-C emission backend with an evidence-grounded generation step. The deterministic pipeline remains responsible for recovering program facts, while the LLM is responsible for rendering those facts into readable pseudo-C, including naming, expression selection, explanatory comments, and explicit handling of unresolved context.

In this thesis, the generation stage is instantiated with Claude Opus 4.6. The model's role is to combine the available skeleton evidence into a readable pseudo-C representation. The model is therefore used as an evidence-grounded emitter rather than as a primary program analyser.

The faithfulness constraints are operational rather than formal guarantees. They are enforced at the interface level by the prompt template and the required output schema. The prompt states that deterministic evidence has priority over semantic enrichment: generated statements must be traceable to recovered instructions, control-flow regions, trigger records, or explicit context records. The model is instructed not to replace mechanical facts recovered by the deterministic pipeline with unsupported high-level interpretations. When a semantic name or contextual explanation cannot be supported by the evidence package, the output is required to preserve the lower-level representation or mark the uncertainty explicitly.

These constraints do not guarantee that the LLM will always produce a faithful output. Instead, they define the intended contract between the deterministic pipeline and the generation stage. The extent to which the generated pseudo-C satisfies this contract is evaluated empirically in Chapter 5 using operation-level consistency, control-flow fidelity, and semantic-token preservation.

Chapter 4

Implementation

4.1 Implementation Overview and Scope

This chapter describes the implemented prototype of the BFA decompilation pipeline introduced in Chapter 3. The prototype implements the workflow from a textual BFA source routine to a generated pseudo-C output: the source routine is parsed, canonicalised, converted into a typed control-flow graph, lifted into an intermediate representation, summarised into a structured evidence skeleton, and finally passed to an LLM-based pseudo-C generation interface. **The implementation therefore covers the complete research pipeline evaluated in this thesis**, from deterministic evidence recovery to pseudo-C generation.

The implementation is not a general-purpose binary decompiler. Its input is a textual BFA source artefact from the studied codebase, after the relevant build variant, macro expansion, and include context have been resolved by the frontend. The implemented parser handles the language constructs needed by the evaluated dataset, including labels, VLIW-style instruction bundles, instruction operands, predicates, memory references, macro-expanded instruction sequences, direct branches, dispatch-like control transfers, termination effects, and communication operations. The implementation also includes bounded repository-context extraction for selected surrounding artefacts, such as headers, caller-side metadata, sibling routines, symbolic constants, memory layout information, and communication metadata when these can be recovered statically.

The deterministic part of the implementation produces a JSON skeleton that serves as the interface between program analysis and generation. This skeleton contains the canonical instruction stream, basic blocks, typed CFG edges, lifted IR statements, structured control-flow evidence, trigger records, control-flow summaries, source-map information, and selected contextual metadata. The generation interface then trims this skeleton, inserts the retained evidence into a fixed prompt template, invokes the LLM, and expects a structured output containing pseudo-C, supporting notes, and unresolved gaps.

Several limitations follow from this scope. First, the implementation does not decode native binaries and does not attempt to support arbitrary instruction sets. Second, instruction lifting is implemented for the instruction forms encountered in the evaluated BFA routines; unknown or unsupported instructions are preserved as unresolved evidence rather than silently discarded. Third, context extraction is intentionally bounded and depends on the repository layout and naming conventions available in the studied industrial codebase. Fourth, the generated output is pseudo-C

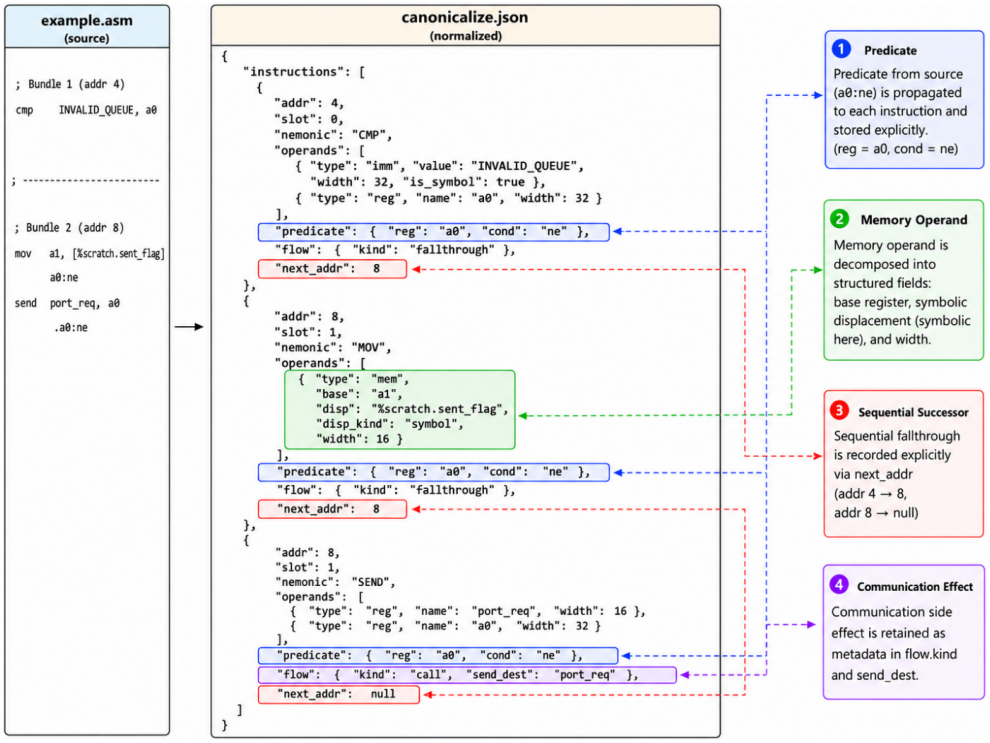


Figure 4.1: Frontend transformation from BFA source to canonical JSON. The parser preserves the source-level bundle structure, while the canonicalisation pass rewrites instructions into normalised records with explicit operands, predicate metadata, control-flow information, and bundle-level fallthrough addresses.

for program comprehension, not guaranteed compilable C. Finally, as discussed in Section 4.7.2, the implementation does not include a formal post-generation verifier; the faithfulness of the generated pseudo-C is assessed empirically by the evaluation framework in Chapter 5.

4.2 Frontend Implementation: Parsing and Canonicalisation

4.2.1 Frontend Data Records

The frontend consists of a parser followed by a canonicalisation pass. The parser consumes the BFA lexical forms introduced in Table 2.1 and emits a bundle-level parser AST. The canonicalisation pass then rewrites this AST into the canonical instruction records stored in the JSON skeleton. Figure 4.1 illustrates this transformation on a simplified BFA example: source-level bundles are preserved in the parser AST and then normalised into instruction records with explicit operands, predicates, control-flow metadata, and fallthrough addresses.

The parser output is a flat AST with three implementation-level node types: *Label* nodes for symbolic source positions, *Bundle* nodes for VLIW issue groups, and *MacroCall* nodes for inline macro invocations. This representation is deliberately close to the surface structure of the source: labels, bundles, and macro calls remain visible rather than being immediately lowered into control-flow or data-flow constructs.

Each bundle receives a synthetic address, and labels inherit the address of the immediately following bundle. This addressing scheme is an implementation choice

rather than an architectural machine address. It gives later stages stable identifiers for fallthrough computation, branch-target resolution, source mapping, and CFG construction, while preserving the fact that all instructions in the same bundle belong to the same issue group.

The canonicalisation pass converts the parser AST into typed frontend records. The output preserves bundle structure, but each instruction is rewritten with normalised operands, predicate metadata, a canonical control-flow kind, and an explicit fallthrough address. Table 4.1 summarises the main fields used in the JSON skeleton.

Table 4.1: Main fields in the frontend JSON records.

Field	Scope	Description
`type`	AST node	Node type, such as `Bundle`, `Label`, or `MacroCall`.
`addr`	Bundle / instruction	Synthetic bundle-level address shared by all instructions in the same bundle.
`next_addr`	Instruction	Address of the next bundle in source order, or `null` at the end of the program.
`parallel`	Bundle	Boolean indicating whether the bundle contains multiple parallel operations.
`inline_macros`	Bundle	Inline macro invocations associated with the bundle, if any.
`mnemonic`	Instruction	Normalised instruction mnemonic.
`operands`	Instruction	Array of typed operand records, such as register, immediate, memory, label, or condition operands.
`width`	Instruction / operand	Inferred operation or operand width in bits.
`predicate`	Instruction	Instruction-level guard condition, if present.
`flow.kind`	Instruction	Normalised control-flow category; see Table 4.3 for the mapping.
`flow.targets`	Instruction	Resolved branch, dispatch, or communication targets.

4.2.2 Bundle-Level Addressing and Fallthrough

As discussed in Section 2.1.1, BFA programs are organised around VLIW-style bundles rather than a purely sequential instruction stream. The frontend therefore assigns addresses at bundle granularity. During parsing, a synthetic address counter is incremented once per bundle, and all instructions inside the same bundle share the same `addr`. These addresses are internal identifiers used for branch-target resolution, source mapping, and CFG construction; they are not architectural machine addresses. This design prevents parallel instructions from being misinterpreted as a sequential chain.

The same bundle-level convention is used for fallthrough. The canonicalisation pass computes a `next_addr` field for each instruction, but this field points to the next bundle rather than to the next instruction in the same bundle. As a result, all instructions in one bundle receive the same `next_addr`. Labels are also mapped to bundle addresses, since a label marks a source position before a bundle rather than an executable operation. The representation of labels in the canonical output is illustrated in Figure 4.1, while Figure 4.2 shows the difference between bundle-level and instruction-level fallthrough.

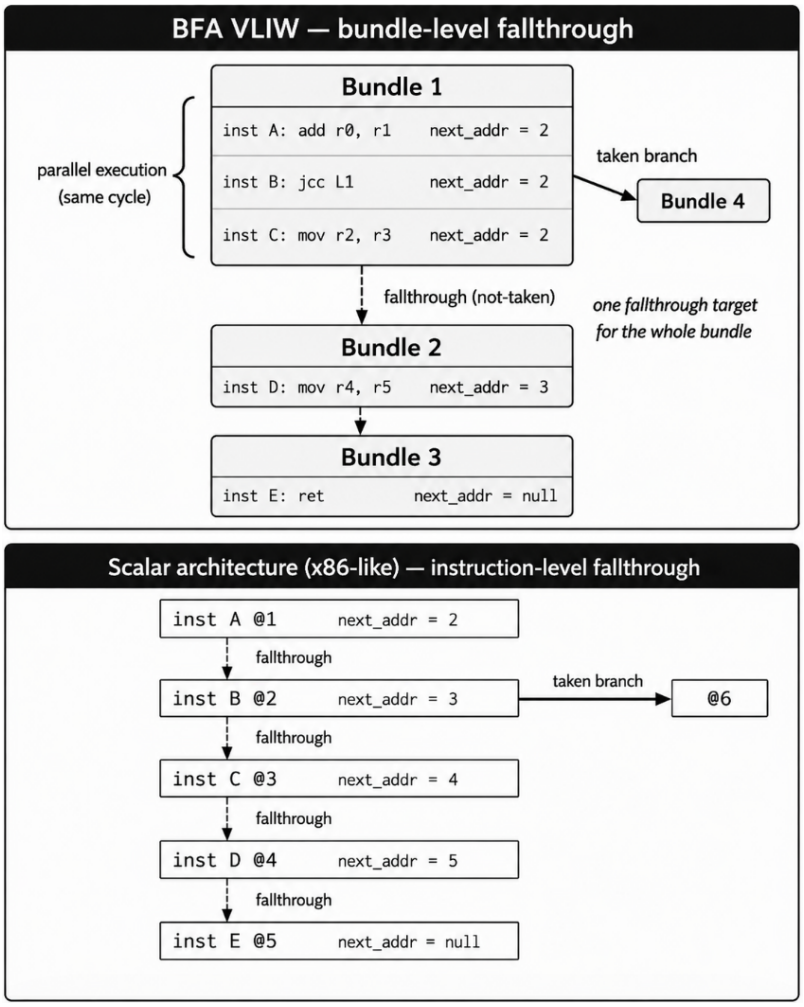


Figure 4.2: Bundle-level fallthrough compared with instruction-level fallthrough in a scalar x86-like sequence. In the VLIW case, all instructions in the same bundle share one `next_addr`; for a conditional branch, the not-taken successor is the next bundle rather than the next sibling instruction.

4.2.3 Operand and Flow-Kind Normalisation

The canonicalisation pass normalises parser-level operands into typed canonical records, but its main role is also to enrich these records with architectural metadata. Register operands are resolved against a register-width and register-class map derived from the architecture reference manual. This allows the frontend to attach bit widths, sub-register access information, and coarse execution roles to operands that were previously represented only as source tokens. Memory operands are similarly interpreted using the role of their base registers and symbolic displacements.

Table 4.2 summarises the register categories used to attach width and role metadata. The purpose of this metadata is similar to an x86 frontend distinguishing full registers, partial registers, addressing registers, and condition flags before lifting instructions into an intermediate representation.

Table 4.2: Register categories used during operand normalisation.

Category	Examples	Width	Canonical role
Accumulator-style registers	`acc0–acc7`	`64-bit`	Arithmetic datapath state used for intermediate values, arithmetic results, and wide computations.
Address registers	`r0–r3`	`64-bit`	Base or index state used to construct memory references and address calculations.
Message or token registers	`msg0–msg3`	`32-bit`	Payload state used by communication, dispatch, or message-passing operations.
Predicate and flag state	`zf`, `cf`, `p0`	`1-bit`	Boolean condition state used for branches and instruction-level guards.
Special-purpose registers	`pc`, `sp`, `ctx`	`64-bit`	Execution-context state with fixed roles, such as control position, stack-like state, or runtime context.

The same pass also maps parser-level control-flow annotations to canonical `flow.kind` values. This mapping gives the CFG builder a small and uniform vocabulary for local control-flow effects, instead of requiring it to reinterpret instruction mnemonics directly. Table 4.3 shows the mapping used by the frontend.

Table 4.3: Control-flow annotation and canonical flow-kind mapping.

Instruction pattern	Parser annotation	Canonical kind	CFG evidence
Unconditional branch	`jmp`	`jmp`	Single successor: the branch target.
Conditional branch	`conditional_jmp`	`jcc`	Two successors: the branch target and `next_addr`.
Switch-like dispatch	`switch`	`ijmp`	Multiple successors: the dispatch targets.
Communication or dispatch operation	`send`	`call`	Observable dispatch effect with a recorded destination target.
Explicit termination	`exit`	`ret`	No ordinary successor.
Ordinary instruction	`fallthrough`	`fallthrough`	Single successor: `next_addr`.

4.2.4 Predicate Propagation

BFA supports instruction-level predication through `IF/ELSE` constructs. Unlike a conditional branch, which changes the program counter, an `IF` construct acts as an execution mask over instructions in the current bundle. Guarded instructions are conditionally executed, but control still falls through to the next bundle unless an explicit control-flow instruction is present.

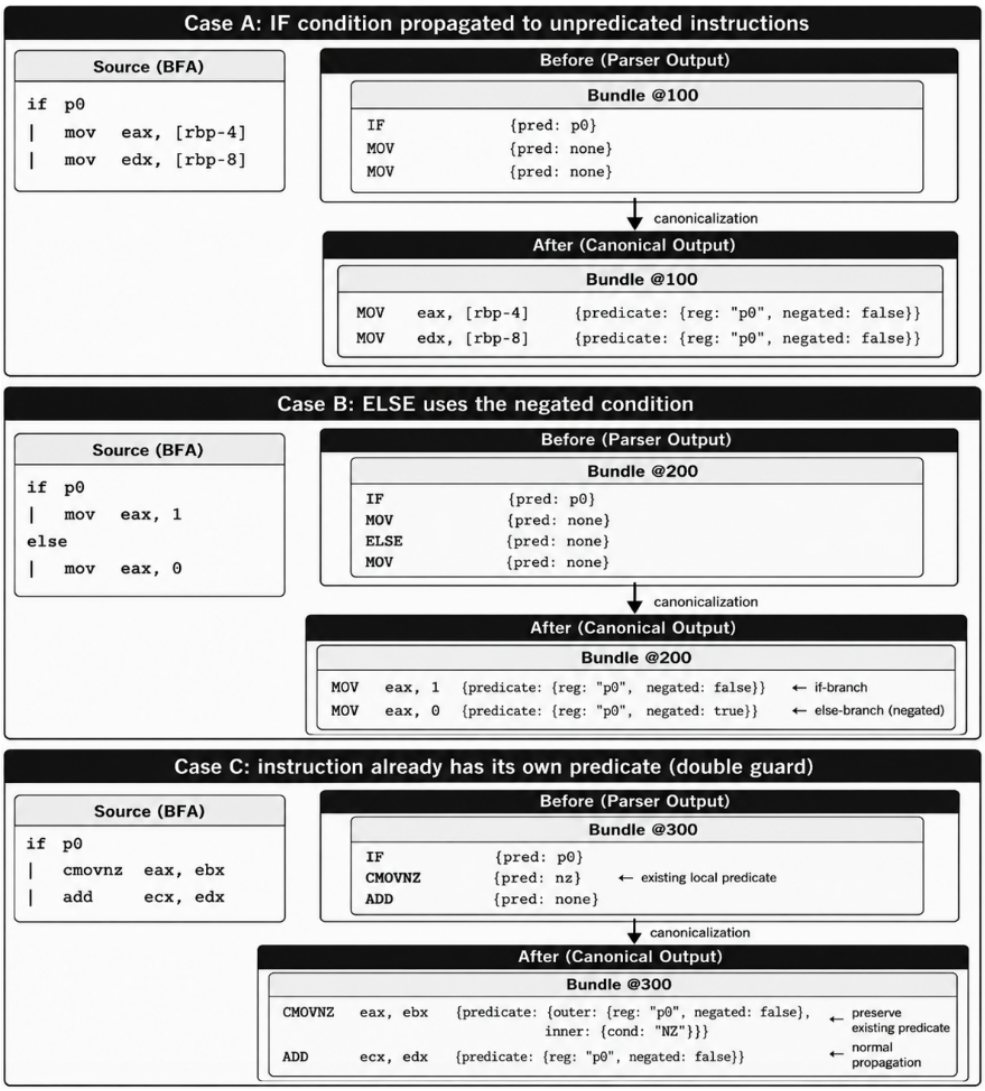


Figure 4.3: Predicate propagation during canonicalisation.

At the parser-AST level, this relationship is still implicit: the `IF` or `ELSE` marker may appear as a standalone bundle entry, while the sibling instructions do not yet carry explicit predicate metadata. The canonicalisation pass makes this dependency explicit by propagating the active condition to the affected instruction records. Instructions guarded by `IF` receive the original condition; instructions following an `ELSE` marker receive its negation. The marker instructions themselves are removed from the canonical instruction list.

If an instruction already has its own predicate, the propagated bundle-level condition is preserved as an outer guard rather than overwriting the local predicate. This keeps the combined execution condition explicit. After this step, later stages can distinguish predicated execution from branch-based control flow: only instructions with an explicit branch `flow.kind` create CFG successors, while predicated non-branch instructions remain within the current basic block. Figure 4.3 illustrates the three propagation cases: a plain `IF` guard applied to unpredicated instructions, an `ELSE` region represented by the negated guard, and an instruction that already carries its own local predicate.

4.3 CFG and Structural Recovery Implementation

The canonical instruction sequence produced by the previous phase 4.2 is consumed by this control-flow recovery stage. This stage first constructs a block-level control-flow graph from the canonical instruction stream, and then transforms the graph into a structured abstract syntax tree. The implementation separates these two responsibilities: CFG construction recovers basic blocks and typed control-flow edges, while structural recovery interprets the resulting graph in terms of higher-level control constructs.

4.3.1 Basic Block Construction

Basic block construction consists of leader detection followed by block partitioning. In the classical compiler formulation, a basic block is a maximal straight-line region with a single entry point and a single exit point, and a control-flow graph (CFG) represents basic blocks as nodes and possible transfers of control as directed edges [Aho et al., 2007; Allen, 1970]. Decompiler frontends use the same general construction before applying later control-flow and data-flow analyses [Cifuentes, 1994].

This thesis does not introduce a new basic-block construction algorithm. Instead, it adapts the standard leader-based construction to the BFA execution model. Since BFA instructions are organised into bundles, the atomic unit for block construction is the bundle rather than the individual scalar instruction. A basic block is therefore treated as a maximal sequence of bundles with a single entry point and a single block-level exit.

Leader Detection. Leader detection follows the standard leader-based CFG construction method: the entry instruction is a leader, branch targets are leaders, and the fallthrough instruction after a control-transfer instruction is also a leader [Aho et al., 2007; Cifuentes, 1994]. In this implementation, the same rules are applied over bundle-level addresses in the canonical instruction stream. Since all instructions in the same bundle share one `addr`, the implementation marks leader *addresses* rather than individual scalar-instruction positions. The leader set is constructed using three rules:

(a) The address of the first bundle in the routine is always marked as a leader.

(b) Every resolved branch or dispatch target is marked as a leader, because control may enter the routine at that address from a non-sequential predecessor.

(c) For every instruction whose `flow.kind` represents an explicit control-flow transfer, such as a conditional branch, an unconditional branch, an indirect or multi-way dispatch, or a return/termination effect, the corresponding fallthrough successor `next_addr` is marked as a leader when it exists. In addition, the address of the bundle containing the control-flow instruction is marked as a leader. This isolates terminator bundles as block boundaries while preserving the bundle as the atomic unit of control-flow construction.

Block Partitioning. Block partitioning also follows the standard leader-based construction: once the leader set has been identified, consecutive leaders delimit the basic-block regions [Aho et al., 2007]. In the BFA-specific implementation, the collected leader addresses are sorted in ascending order, and consecutive leaders define

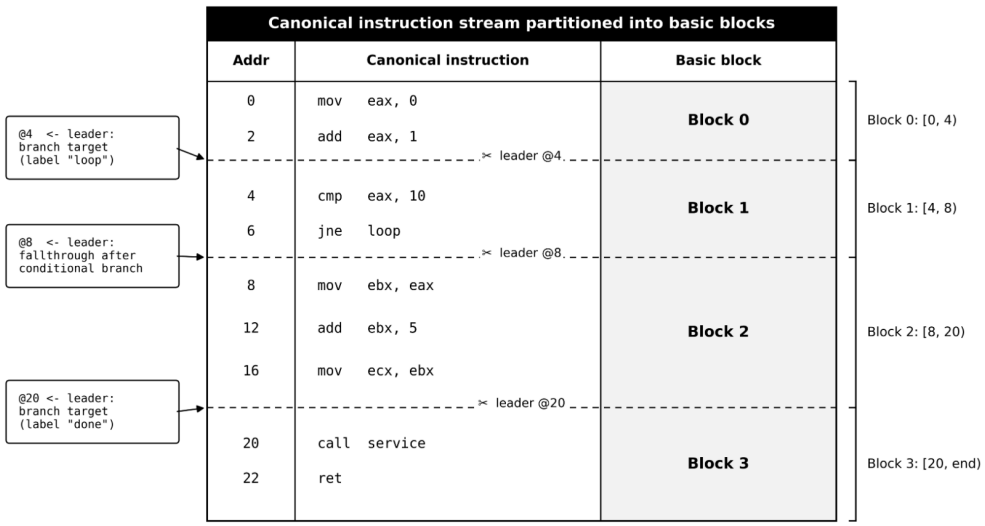


Figure 4.4: Instruction stream partitioned into basic blocks by leader addresses. Branch targets and fallthrough successors after control-flow instructions become leaders, and consecutive leaders define half-open block intervals.

half-open intervals $[L_i, L_{i+1})$, each of which is instantiated as one basic block. The final leader defines the last interval, extending to the end of the canonical instruction stream. Since addresses are assigned at bundle granularity, every bundle whose `addr` falls within an interval is assigned to the corresponding block, and instructions in the same bundle are never split across blocks.

The blocks are then assigned sequential identifiers according to address order. Labels and branch targets are resolved to the block beginning at the corresponding leader address. Empty blocks may be retained when a label has no intervening executable bundle, preserving the label-to-block mapping required for later edge construction. Figure 4.4 illustrates how leader addresses partition a flat canonical stream into basic block intervals.

4.3.2 Typed Edge Construction

After basic block construction, the CFG builder constructs the outgoing edges of each block. The implementation scans the instructions contained in a block and selects the last instruction whose `flow.kind` is not ordinary fallthrough as the block terminator. The terminator's `flow.kind`, resolved target information, and pre-computed `next_addr` determine the successor set of the block. If no explicit terminator is present, the block is connected to the next block in address order by an ordinary fallthrough edge, when such a successor exists.

Each emitted edge is annotated with one of the generic edge kinds listed in Table 4.4. Conditional branches produce separate taken and not-taken edges. The taken edge points to the resolved branch target, while the not-taken edge usually corresponds to the fallthrough successor. Unconditional branches produce a single target edge. Indirect or multi-way dispatch instructions produce one edge for each resolved case target and, when available, a default edge. Return or termination instructions are represented as terminal control-flow transfers and do not introduce ordinary fallthrough successors.

The purpose of these annotations is to preserve the role of each transfer of control,

Edge kind	Meaning
TRUE_BRANCH	Edge taken when a conditional branch condition evaluates to true.
FALSE_BRANCH	Edge taken when a conditional branch condition evaluates to false, normally corresponding to the fallthrough successor.
FALLTHROUGH	Sequential transfer to the next block without an explicit control-flow instruction.
UNCONDITIONAL	Edge produced by an unconditional branch to a resolved target.
SWITCH_CASE	Edge to one resolved case target of an indirect or multi-way dispatch.
DEFAULT	Default dispatch edge used when no explicit case target matches.
EXIT	Terminal edge produced by a return or program-termination instruction.

Table 4.4: Typed CFG edge kinds used during edge construction.

rather than only recording graph connectivity. This information is consumed by the later structuring pass, which uses typed edges to distinguish conditional regions, unconditional jumps, multi-way dispatches, default paths, and terminal exits before rewriting the CFG into a structured AST [Cifuentes, 1994; Yakdan et al., 2015].

4.4 IR Lifting Implementation

The IR lifting stage translates the canonical instructions inside each basic block into a semantic intermediate representation. The pass operates locally within blocks: it replaces low-level instruction records with typed IR statements, while preserving the inter-block CFG topology, typed edges, and terminator information constructed in the previous stage. The output is an IR-CFG in which each block contains both its lifted data-flow representation and its original control-flow connectivity.

The lifted IR uses SSA-style temporaries to make intermediate data dependencies explicit [Cytron et al., 1991]. Implicit architectural state is represented through explicit state-access operations, such as reading from or writing to a register-like state field. This design allows later stages to query data-flow and control-flow information from a single structured object without reinterpreting the original instruction syntax.

4.4.1 IR Data Model

The IR data model consists of typed values and typed instructions. Values are represented using four main forms: constants, SSA-style temporaries, architectural state fields, and memory addresses. Constants represent literal values with optional bit-width metadata. Temporaries represent intermediate SSA values created during lifting. State fields represent register-like architectural locations, while address values represent typed memory references with a base and displacement.

Each IR instruction records an opcode, a list of operands, an optional result temporary, and provenance metadata linking the IR statement back to the canonical instruction from which it was derived. The opcode set is intentionally small and grouped by semantic role, and due to confidentiality reason the detailed instruction-level classification for operation type could not be presented here. Instructions that cannot be resolved by the lifter are represented conservatively using an unknown

Source–destination pattern	Actual semantics	Generated IR pattern
reg → reg	Register assignment	READ_STATE → WRITE_STATE.
mem → reg	Memory read	LOAD → WRITE_STATE.
reg → mem	Memory write	READ_STATE → STORE.
imm → reg	Immediate load	WRITE_STATE(Const).

Table 4.5: Operand-dependent lowering cases for the MOV instruction.

or unresolved opcode, rather than being silently discarded.

4.4.2 Instruction Translation Dispatch

We implement instruction lowering through a mnemonic-based dispatcher. For each canonical instruction, the dispatcher selects a dedicated lowering function according to the normalised mnemonic. Each lowering function emits a sequence of IR instructions that captures the observable effect of the source instruction using the opcode categories defined in the IR data model.

However, a single syntactic instruction form may produce different IR patterns depending on operand types. The MOV instruction is a representative case: its surface form is uniformly `MOV src, dst`, but its semantics depend on whether the operands refer to registers, memory locations, or immediate values. Table 4.5 summarises the main lowering cases.

To avoid duplicating this operand-type logic across all lowering functions, the implementation factors common behaviour into shared utilities. Source operands are first resolved into IR values, destination operands are routed to either state writes or memory stores, and predicate handling is applied uniformly after the base lowering pattern has been constructed. This keeps the dispatcher mnemonic-oriented while still allowing each instruction to produce the correct operand-specific IR sequence.

4.4.3 Bundle-Aware IR Emission

Because BFA uses VLIW-style bundles, instructions in the same bundle cannot be lowered as a simple sequential list. All source operands in a bundle must be read from the state visible at bundle entry, regardless of writes performed by sibling instructions in the same bundle [Fisher, 1983]. A naive sequential lowering would allow a later instruction to observe a value written by an earlier instruction from the same bundle, which would incorrectly introduce an artificial intra-bundle data dependency.

To preserve the parallel read-before-write semantics in a sequential IR, the lifter emits each bundle in two phases. The first phase is the compute phase, which contains source reads, memory loads, constant materialisation, arithmetic operations, comparisons, and other computations that produce temporary SSA values. The second phase is the writeback phase, which contains architectural state writes, memory stores, communication operations, and other observable side effects. Since all reads are emitted before any writeback operation, every instruction in the bundle observes the same pre-bundle state.

Figure 4.5 illustrates the difference between naive sequential lowering and the two-phase lowering used by the implementation. The example shows a bundle that

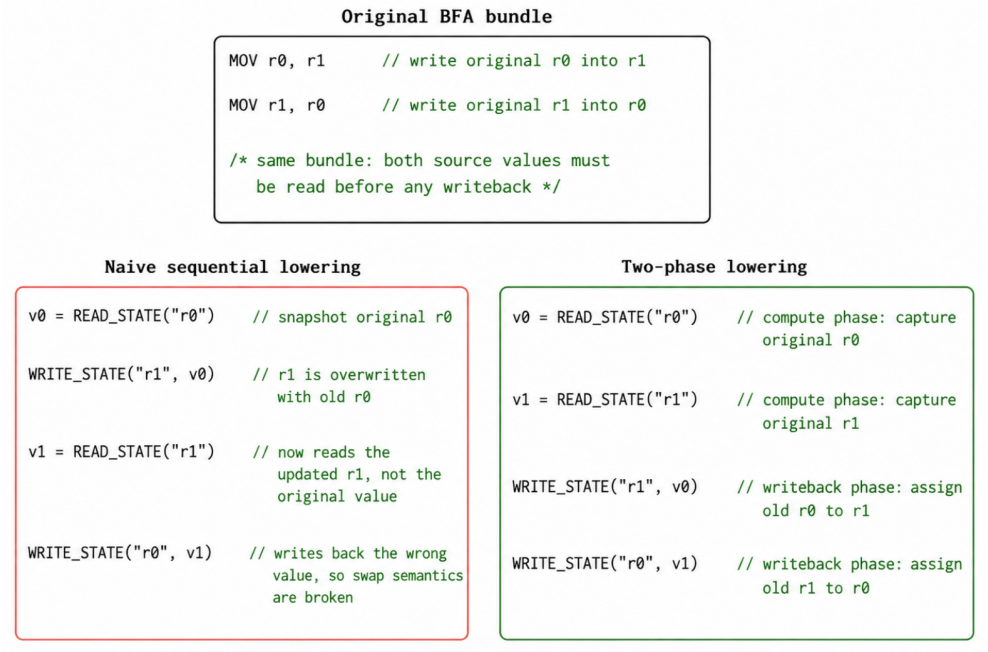


Figure 4.5: Bundle-aware two-phase IR emission.

swaps two register values. Sequential lowering breaks the swap because the second read may observe the first write, while two-phase lowering first captures both original values and only then performs the two writebacks.

4.4.4 Predicated Writes

Predicated instructions are lowered as conditional data-flow updates rather than as additional CFG branches. In this implementation, a *predicate* is a recovered guard condition attached to an instruction during canonicalisation. Such guards may come from explicit instruction-level predicates, bundle-level `IF/ELSE` constructs propagated to the affected instructions, or condition-code tests produced by earlier compare-like operations. After canonicalisation, each predicate is represented as a normalised condition record, for example a condition state together with the expected value or polarity.

When an instruction carries such a predicate, the lifter first lowers the instruction's unguarded computation into temporary IR values. It then evaluates the predicate by reading the relevant condition state and comparing it against the expected value. The result is a boolean guard used by a `SELECT` operation. The predicate is therefore treated as existing control/data-flow evidence from the BFA input, not as a new condition inferred by the lifter.

For a single predicated write to destination d, the generated pattern is:

$$v_{\text{old}} = \texttt{READ_STATE}(d), \qquad v_{\text{out}} = \texttt{SELECT}(p, v_{\text{new}}, v_{\text{old}})$$

followed by:

$$\texttt{WRITE_STATE}(d, v_{\text{out}})$$

Here, p is the evaluated predicate condition, v_{new} is the value computed by the predicated instruction, and v_{old} is the current value of the destination. The `SELECT`

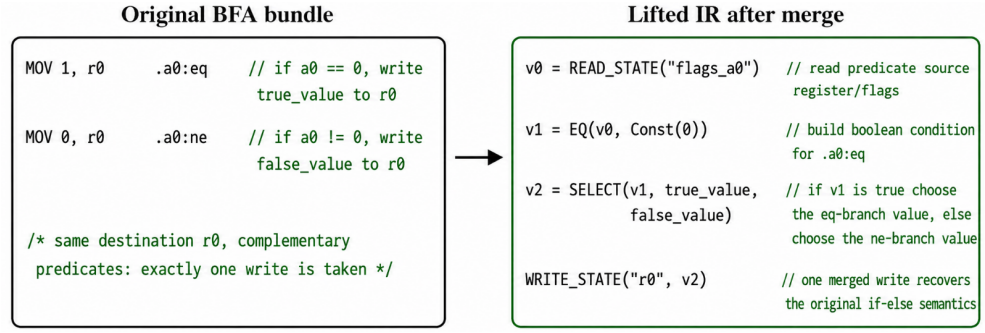


Figure 4.6: Predicated writes merged through a SELECT operation. The example shows two complementary predicated writes to the same destination. The lifter evaluates the predicate once, selects between the two candidate values, and emits a single merged writeback.

operation is analogous to a ternary conditional expression in C and chooses the new value when the predicate is true, or the old value when the predicate is false [Lattner and Adve, 2004].

A closely related case occurs when two complementary predicated writes target the same destination. In this situation, the two candidate values can be merged into a single SELECT, followed by one final writeback. This avoids emitting two separate conditional updates while preserving the original if-else-like semantics, as illustrated in Figure 4.6.

This design preserves the semantics of predicated execution without changing the control-flow graph. Predicated non-branch instructions remain inside the current basic block, and the CFG is not expanded with artificial if-else regions. Conditional execution is therefore represented in the data-flow layer, while branch-based control flow remains the responsibility of the CFG.

4.5 Structured AST Construction

The structured AST construction pass consumes the typed-edge CFG and rewrites it into a hierarchical representation of source-like control flow. The input to this pass is a block-level graph whose nodes contain recovered instructions and whose edges are annotated with the edge kinds defined in Table 4.4. The pass uses this typed graph to recover structured regions such as sequences, conditionals, loops, and multi-way dispatches.

The implementation first computes the graph relationships required for structuring, including predecessor sets, successor sets, dominators, post-dominators, immediate post-dominators, and back edges. Dominance information is used to recognise loop structure, while post-dominance information is used to identify where acyclic branch regions reconverge. For a conditional block with true and false successors, the immediate post-dominator of the branch block is treated as the merge point of the conditional region. During recursive region collection, each branch arm is collected only up to this merge block. The merge block is then emitted after the conditional as sequential continuation code, rather than being absorbed into either branch body.

Figure 4.7 illustrates this process on a representative diamond-shaped CFG. The branch block splits into two regions, which are recovered as the `then` and `else` arms of an `IfElse` node. The reconverging block is identified as the merge point and is

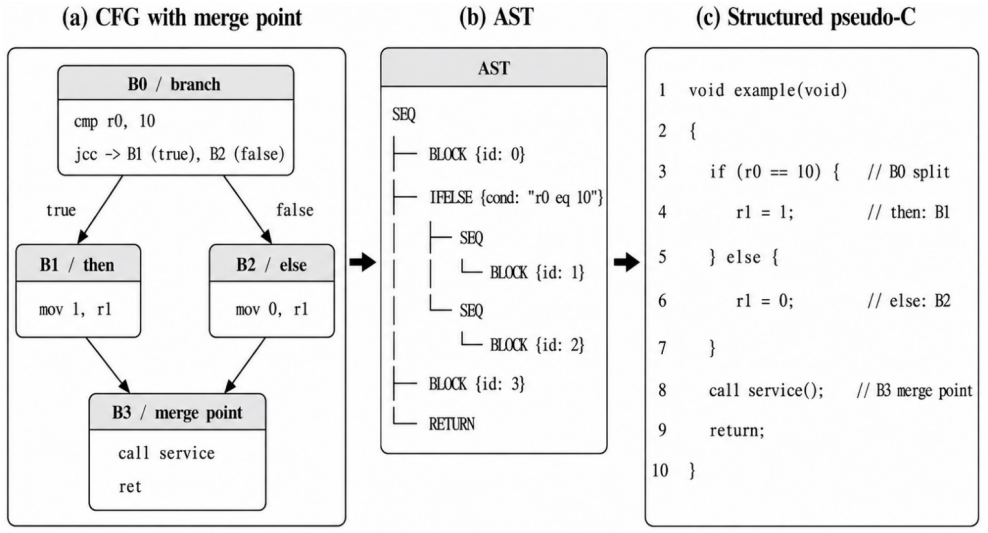


Figure 4.7: Structured AST construction from a typed CFG. The branch block splits into true and false regions, which reconverge at a merge block. The merge block is emitted after the `IfElse` node as sequential continuation code rather than being absorbed into either branch body.

placed after the conditional in the resulting structured AST and pseudo-C output.

Loop regions are identified using dominance-based back-edge detection. An edge from a latch block to a header block is treated as a back edge when the header dominates the latch. The target of the back edge becomes the loop header, and the source becomes the latch. The loop body is collected by walking backwards through predecessor edges from the latch towards the header, accumulating the blocks that belong to the natural loop region [Cifuentes, 1994; Schwartz et al., 2013]. Edges inside the loop that target the header are mapped to `Continue` nodes when possible, while edges leaving the loop body are mapped to `Break` nodes. The resulting cyclic region is emitted as a `While` node when the loop boundary and continuation point can be

After these graph relationships have been computed, the structurer performs region-based AST emission over the typed-edge CFG. This final step uses three pre-computed artefacts: the typed CFG, the dominator and post-dominator relations, and the set of detected natural loops. The typed CFG identifies the role of each transfer, such as true branch, false branch, fallthrough, unconditional jump, switch case, default edge, or exit. The post-dominator relation defines the boundaries of acyclic regions, especially conditional regions. The natural-loop information defines the boundaries of cyclic regions, including loop headers, latches, and loop bodies.

Straight-line regions are emitted as `Seq` nodes. Blocks with true and false branch edges are emitted as `IfElse` nodes, using the post-dominator merge point as the region boundary. Loop regions detected from back edges are emitted as `While` nodes. Multi-way dispatch regions are recovered from `SWITCH_CASE` and `DEFAULT` edges and emitted as `Switch` nodes when the case targets are explicit.

This design differs from structuring over an untyped CFG, where the meaning of each outgoing edge must be inferred from graph topology alone. In this implementation, branch roles have already been made explicit during canonicalisation and typed edge construction. As a result, the region-matching step can combine edge roles, post-dominator boundaries, and natural-loop regions directly, instead of relying only on heuristic pattern matching over a raw CFG. Assuming the typed CFG,

dominance relations, post-dominance relations, and natural loops have already been computed, the final region-emission pass is linear in the number of basic blocks.

The resulting AST uses a small set of node kinds: `Seq`, `Block`, `IfElse`, `While`, `Switch`, `Return`, `Break`, `Continue`, and `Goto`. Each structured node retains links to the underlying basic blocks, so the recovered source-like shape remains traceable to the deterministic CFG evidence. When the pass encounters irreducible or ambiguous control flow, it does not force the region into an incorrect high-level construct. Instead, it preserves the transfer explicitly using continuation logic or a `Goto` node. In particular, a `Goto` node is used as a fallback when a block has already been structured but is reached again through another control-flow path.

The final structured AST is therefore more readable than the raw CFG while remaining grounded in the block and edge evidence recovered by the deterministic frontend.

4.6 Skeleton Packaging and Evidence Extraction

After canonicalisation, CFG construction, IR lifting, and structured AST construction, the pipeline packages the recovered evidence into a single JSON skeleton. This skeleton is the main interface between deterministic analysis and LLM-based pseudo-C generation. It contains the canonical instruction stream, basic blocks, typed CFG edges, lifted IR statements, structured AST nodes, source-map information, and selected derived evidence records.

The purpose of this stage is not to infer new semantics. Instead, it reorganises facts already recovered by the deterministic pipeline into forms that are easier to inspect, cache, and consume during generation. In addition to storing the full low-level artefacts, the implementation derives two compact views from the skeleton: trigger records and a control-flow summary. A bounded context metadata pass also attaches recoverable information from surrounding source artefacts, but unresolved facts are left explicit rather than guessed.

4.6.1 Trigger Record Extraction

The lifted IR-CFG encodes semantic information in SSA-style statements, but important source-level facts may be distributed across several low-level IR operations. For example, a packed-field access may appear as a sequence of state reads, shifts, masks, and writes. Although this representation is faithful at the IR level, it is unnecessarily fragmented for the generation stage. Trigger record extraction therefore collapses recognised multi-statement IR patterns into flat evidence records that the LLM can consume without performing def-use reasoning over the full IR.

Figure 4.8 illustrates this process for a packed-field extraction. The original low-level instructions are first lifted into several SSA-style IR operations. The trigger extractor then recognises the read–shift–mask–write pattern and records it as a single packed-field access. This gives the generation stage direct access to the recovered meaning, while the underlying IR remains available as provenance evidence.

The extractor performs a deterministic pass over the IR blocks and emits records only for patterns that can be recognised directly. Each record stores its source block, originating instruction address, trigger category, relevant operands, and a provenance link back to the canonical instruction or IR statement. Synthetic IR operations introduced only to model predication are filtered out when they do not correspond

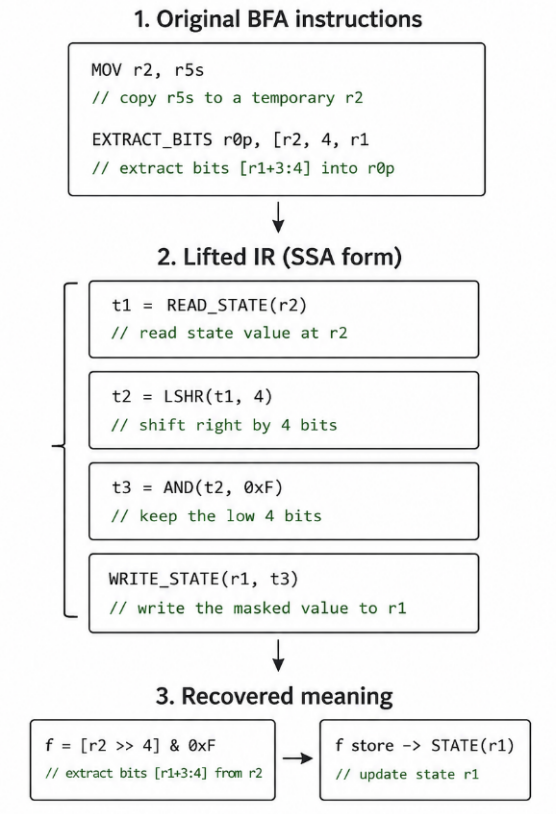


Figure 4.8: Recovering a packed-field access from lifted SSA-form IR. A sequence of READ_STATE, shift, mask, and write operations is recognised as one packed-field access trigger record, allowing the generation stage to consume the semantic meaning directly rather than reconstructing it from def-use chains.

to source-level operations, preventing the generation stage from emitting spurious statements.

The trigger categories are intentionally limited to recurring evidence patterns that are useful for pseudo-C generation. Table 4.6 summarises the categories used by the implementation.

The trigger layer is therefore a compact summary of deterministic evidence, not a replacement for the lifted IR. When a pattern is ambiguous, the original IR statements remain in the skeleton, but no higher-level trigger is invented. This keeps the generation stage grounded in recovered evidence while reducing the amount of low-level symbolic reconstruction expected from the LLM.

4.6.2 Control-Flow Summary

The control-flow summary serialises selected CFG facts into flat queryable records. The structured AST provides the preferred hierarchical view for pseudo-C generation, but it may hide the exact block-level topology behind source-like constructs. The summary therefore preserves the original control-flow evidence, including block identifiers, terminators, typed outgoing edges, branch conditions, dispatch targets, and fallback transfers.

This flat view complements the AST. When a region is successfully structured, the summary still records the original edges and target blocks that justify the structured

Trigger category	Description
`memory_reference`	A recognised load or store through a recovered memory address.
`packed_field_access`	A recognised shift-and-mask or extraction pattern over a packed value.
`symbolic_constant`	A literal or symbolic constant that may support naming or explanation in the generated pseudo-C.
`communication_event`	An observable message-passing or dispatch operation recovered from the IR.
`resource_operation`	An allocation, deallocation, or similar resource-management event.
`termination`	An explicit return, exit, or terminal control-flow event.
`unresolved`	A preserved low-level operation whose higher-level role cannot be resolved deterministically.

Table 4.6: Trigger record categories extracted from the lifted IR.

node. When a region cannot be safely structured and a `Goto` fallback is emitted, the summary preserves the precise successor relationships. Thus, the AST supports readable rendering, while the control-flow summary keeps the generated output grounded in deterministic typed-edge evidence.

4.6.3 Context Metadata Recovery

The local skeleton captures the structure and semantics recovered from the current low-level routine, but it does not always contain all information needed for readable pseudo-C generation. Some operands refer to symbolic constants, memory layouts, packed data formats, communication endpoints, or invocation context that are defined outside the current file. The context metadata recovery step therefore attaches additional repository-derived evidence to the skeleton when such information can be recovered deterministically.

The concrete lookup rules are intentionally described only at a high level, because they depend on the layout, naming conventions, and auxiliary artefacts of the industrial codebase used in this thesis. In general, the implementation starts from the routine being decompiled and searches a bounded set of nearby source artefacts for relevant definitions, symbolic names, type-like layout information, communication metadata, and related routines. Later lookup steps may use facts recovered by earlier steps, but unresolved metadata is left explicitly unresolved rather than inferred.

This context is used only as grounding evidence for pseudo-C generation. It may help the LLM choose clearer names, comments, or field-style references, but it does not override the mechanical facts recovered from the canonical instruction stream, CFG, IR, structured AST, trigger records, or control-flow summary. When a contextual fact cannot be resolved from available artefacts, the skeleton preserves the lower-level representation instead of allowing the model to invent repository-specific semantics.

4.7 LLM Generation Interface

After deterministic evidence recovery, the skeleton already contains the program structure and semantic evidence needed for pseudo-C generation. The LLM is therefore used only as an evidence-constrained emitter, not as a primary program analyser.

It renders the recovered canonical instructions, CFG, IR, structured AST, trigger records, and context metadata into readable pseudo-C.

Compared with a rule-based pretty-printer, the LLM provides more flexibility in naming, expression selection, and explanatory comments. However, this enrichment is strictly additive: the model may clarify recovered behaviour, but it may not invent operations, omit instructions, or override deterministic evidence. When context is missing, the output preserves lower-level names or unresolved values rather than guessing unsupported semantics.

4.7.1 Input Trimming

Before invoking the LLM, the implementation derives a trimmed prompt payload from the full skeleton. The full skeleton remains the authoritative artefact, while the trimmed payload contains only the evidence needed for pseudo-C generation.

The trimming step removes or compresses intermediate data that has already been summarised by later records, such as trigger records, control-flow summaries, and provenance links. It also serialises retained evidence in compact forms, including block summaries and flat evidence lists. This process is conservative. Evidence that affects correctness, such as instruction coverage, branch structure, side effects, unresolved operands, and fallback control flow, is retained. Thus, trimming reduces token cost without changing the deterministic evidence available to the generation stage.

4.7.2 Prompt Template Instantiation

The prompt template is designed around evidence-constrained pseudo-C generation. The model is instructed to treat the deterministic skeleton as the authoritative source of program facts and to use semantic enrichment only as an additive rendering step. Generated statements must be traceable to recovered instructions, lifted IR statements, structured control-flow regions, trigger records, or explicit context metadata. If the evidence is insufficient to support a high-level interpretation, the output must preserve the lower-level form and mark the uncertainty explicitly.

These prompt-level constraints are not treated as a formal guarantee of correctness. The implementation does not include an automatic post-generation verifier that proves equivalence between the generated pseudo-C and the skeleton. Instead, the prompt restricts the model's role before generation, while the actual degree of faithfulness is assessed empirically by the evaluation framework in Chapter 5. In particular, the evaluation compares the generated pseudo-C with skeleton-derived evidence along operation-level, control-flow-event, and semantic-token dimensions.

The prompt is instantiated by inserting the trimmed skeleton into a fixed template. The template defines the model's role, the allowed use of evidence, the priority of mechanical facts over readability-oriented naming, and the required output format. It also provides reconstruction rules that mirror the deterministic pipeline: inspect available evidence, reconstruct instruction effects, respect predicate and bundle boundaries, follow recovered control flow, and only then apply semantic naming or commentary.

For confidentiality reasons, the complete production prompt is not reproduced verbatim. It contains architecture-specific register names, type aliases, instruction conventions, and repository-derived metadata. Table 4.7 summarises the sanitised prompt-template layers used by the implementation. The table gives concrete ex-

Table 4.7: Sanitised prompt-template layers for evidence-constrained pseudo-C generation.

Layer	Purpose	Sanitised example
Role and task boundary	Defines the model as an evidence-grounded pseudo-C emitter rather than an unconstrained translator.	Convert an analysis JSON into one C-like pseudo-code function; do not summarise the routine or infer unsupported business logic.
Negative constraints	Prohibits invented operations, omitted effects, and plausible-looking code that is not grounded in recovered evidence.	If there is no store to `[ctx + flag]`, the model must not emit `ctx.flag = 0;` only because such an initialisation would look natural.
Evidence priority	Defines how to arbitrate between instruction evidence, lifted IR, trigger records, structural hints, and contextual metadata.	If the IR records a load from `[ctx + counter]` into `eax`, the output may name the value `counter`, but it must still emit a load rather than a store.
Reconstruction procedure	Guides evidence inspection, instruction reconstruction, control-flow rendering, and semantic naming.	If a block updates `eax` and then branches on `ZF`, the assignment is emitted before the `if` statement because block statements execute before the terminator.
Prompt-level self-checks	Asks the model to review the output against the prompt constraints before returning it. These are generation instructions, not an automatic verifier.	The prompt asks the model to check that each recovered instruction has a pseudo-C statement, each guarded instruction keeps its guard, and each external event remains visible.
Output contract	Specifies the required structured output fields and pseudo-C formatting rules.	The model returns a structured object containing the pseudo-C body, concise supporting facts, and unresolved gaps such as unknown field names or ambiguous register roles.

amples of prompt-level constraints, but these should not be interpreted as an independent verification mechanism.

The prompt-level self-checks in Table 4.7 should not be interpreted as an enforcement mechanism. They are instructions included in the generation template to reduce common LLM failure modes, such as omitted side effects, widened predicates, unsupported semantic names, or invented operations. Since the model may still violate these instructions, the thesis evaluates the generated output afterwards using the metrics defined in Chapter 5.

4.7.3 Anonymised C-Like Decompilation Output

Listing 4.1 presents anonymised C-like decompilation output produced by the implemented pipeline. The output has been anonymised by replacing proprietary project-specific identifiers with descriptive names. The generated operation structure, control-

flow constructs, semantic values, and explanatory comments have been retained.

The comments shown in the listing form part of the output generated by the LLM and were not added separately for presentation in this thesis. They explain how individual pseudo-C statements relate to recovered state operations, control-flow structure, and communication effects. Because the identifiers have been anonymised, the names shown in the listing should not be interpreted as the original internal project identifiers.

Listing 4.1: Anonymised generated C-like output with LLM-generated comments.

```c
/* Generated pseudo-C routine corresponding to one BFA
   instruction-containing routine. */
void process_control_message(void)
{
    /* Stores a U32 value read from a persistent state field. */
    U32 status;

    /* Stores the bitfield value extracted from status. */
    U32 mode;

    /* Stores the current value of a state counter. */
    U32 counter;

    /* Read the value of STATE_STATUS from the persistent
       execution state. */
    status = READSTATE(STATE_STATUS);

    /* Extract the two-bit mode field starting at bit
       position 4. */
    mode = (status >> 4U) & 0x03U;

    /* Reconstruct the conditional branch found in the
       BFA control-flow graph. */
    if (mode == MODE_ACTIVE)
    {
        /* Read the current counter value from
           STATE_COUNTER. */
        counter = READSTATE(STATE_COUNTER);

        /* Reconstruct the BFA arithmetic operation that
           increments the counter. */
        counter = counter + 1U;

        /* Write the updated counter value back to
           persistent state. */
        WRITESTATE(STATE_COUNTER, counter);

        /* Select the message destination port preserved
           from the BFA routine. */
        SEND(
            PORT_CONTROL,

            /* Preserve the message identifier used by
               the original send instruction. */
            MESSAGE_UPDATE_REQUEST,

            /* Use the updated counter value as the
               message payload. */
            counter
        );
```

```
51      }
52      else
53      {
54          /* Represent the false successor of the
55             reconstructed conditional branch. */
56
57          /* Reset STATE_COUNTER when the active-mode
58             condition is not satisfied. */
59          WRITESTATE(STATE_COUNTER, 0U);
60      }
61  }
```

The anonymised output first uses `READSTATE` to obtain a value from persistent execution state. A shift-and-mask expression then reconstructs the extraction of a two-bit field from the retrieved value. The generated `if`/`else` statement represents a conditional branch recovered from the control-flow graph. On the true branch, another state value is read, incremented, and written back using `WRITESTATE`, after which a message is emitted using `SEND`. On the false branch, the state counter is reset.

The generated comments provide an additional readability layer. They describe the roles of the recovered variables, identify the evidence represented by each state operation, and explain the reconstructed control-flow and communication behaviour. These comments are generated by the same LLM invocation as the pseudo-C statements and therefore form part of the decompiler output rather than a separate manual annotation.

This output contains the three forms of fidelity evaluated in this thesis. The `READSTATE`, `WRITESTATE`, and `SEND` statements represent operation-category preservation. The `if`/`else` construct and the ordering of statements represent control-flow preservation. The state-field identifiers, message port, message identifier, shift value, mask, and integer constants represent the semantic tokens evaluated against the skeleton-derived evidence.

Chapter 5

Evaluation

To evaluate whether the generated pseudo-C preserves the semantics of the original BFA program, this work defines three complementary evaluation axes. The first axis measures instruction-level operation consistency, the second axis measures preservation of the program's control-flow skeleton, and the third axis measures preservation of concrete semantic tokens such as constants, register-derived values, and field-like operands.

- **Axis 1: State Consistency.** This axis asks whether the generated pseudo-C emits the correct kind of operation for the operations present in the BFA input.
- **Axis 2: Control-Flow Graph Fidelity.** This axis asks whether the generated pseudo-C preserves the structural skeleton of the original program, including branches, switches, and externally visible control-transfer events.
- **Axis 3: Semantic-Token Matching.** This axis asks whether the generated pseudo-C preserves concrete semantic values, such as constants, registers, offsets, and symbolic operands.

The three axes are designed to evaluate different aspects of faithfulness. State consistency focuses on local operation types, control-flow fidelity focuses on structural preservation, and semantic-token matching focuses on concrete values and operands. Together, they provide a more task-specific evaluation than surface-level text similarity metrics alone.

5.1 Benchmark Suite

The evaluation is conducted on a benchmark suite of 527 concrete BFA functions drawn from the studied industrial baseband firmware codebase. The benchmark suite consists of naturally occurring BFA routines rather than synthetic examples, and it was not manually selected according to expected model performance. The final suite should be interpreted as the instruction-containing and end-to-end evaluable set of the current prototype: each included routine contains executable BFA instructions and was successfully processed through source discovery, preprocessing, parsing, canonicalisation, CFG construction, IR lifting, structured control-flow recovery, skeleton packaging, pseudo-C generation, and metric extraction.

Each benchmark entry corresponds to one concrete preprocessed BFA routine, rather than necessarily to one unique raw source file. This distinction is important because some BFA source files are compiled under different build or hardware-variant

configurations. Conditional compilation may therefore expose different code paths for different targets. When such variants produce distinct concrete routines, they are evaluated as separate benchmark entries because they exercise different recovered instruction streams, control-flow structures, and semantic tokens.

The benchmark suite was constructed in four stages. First, BFA source artefacts were discovered by traversing the relevant source tree. Second, artefacts that did not contain executable BFA instructions were removed before evaluation. These excluded artefacts were placeholder files rather than failed decompilation cases: they did not provide an instruction stream from which operation types, control-flow events, or semantic tokens could be extracted. Third, each remaining instruction-containing routine was processed by the deterministic frontend and middle-end of the decompiler to produce a canonical JSON skeleton. Fourth, the LLM-based generation stage produced a corresponding pseudo-C output, and a pairing step matched each generated pseudo-C output with its original BFA source and skeleton-derived ground-truth evidence. No instruction-containing routine was excluded because of end-to-end decompilation failure. The only excluded artefacts were placeholder or non-instruction files, which did not contain executable BFA instructions and therefore could not support operation-type, control-flow, or semantic-token evaluation.

The results reported in Chapter 6 therefore measure pseudo-C fidelity over the instruction-containing routines that form the end-to-end evaluable benchmark suite. They should not be read as measuring placeholder artefacts in the source tree, nor as evaluating files that contain no executable BFA instructions.

5.1.1 Computation and Language-Construct Coverage

The benchmark suite covers the main computation patterns and language constructs that the decompiler is intended to handle. These include straight-line data movement, memory loads and stores, arithmetic and bit-level computation, predicated execution, bundle-level parallelism, conditional branches, loop-like control flow, switch-like dispatch, communication events, hardware-job interaction, and suspend/resume-style execution points.

Table 5.1 summarises the main construct categories observed in the benchmark suite. The counts are reported at function level: a function is counted in a row if it contains at least one occurrence of the corresponding construct. The categories are not mutually exclusive, since one function may contain memory access, predication, dispatch, and communication events simultaneously.

Pattern	Functions	Coverage	Role in the benchmark suite
Load/store operations	450	86%	Memory access through scratch, token, and configuration-like regions.
Communication events	399	76%	Externally visible `SEND`-style message dispatch.
Loop-like control flow	341	65%	Backward edges and structured loop recovery.
Switch-like dispatch	216	41%	Multi-way routing, state transitions, and dispatch control flow.
Conditional branches	172	33%	Conditional divergence and `if`/`else` recovery.
Predicated execution	167	32%	Guarded VLIW-style operations without necessarily creating CFG branches.
Bitfield manipulation	162	31%	Shift, mask, extraction, insertion, and packed-field operations.
Hardware accelerator dispatch or response	90	17%	Programming or responding to specialised compute or DMA-like jobs.
Timer programming	67	13%	Timer setup, cancellation, and supervision logic.
Token-register access	290	56%	Inter-processing-element data passing through token-like state.
Event subscription	27	5%	Queue or event-registration operations.
Suspend/resume execution points	404	77%	Cooperative multitasking and non-entry execution contexts.

Table 5.1: Benchmark-suite coverage by language construct and computation pattern. Counts are function-level counts and categories are not mutually exclusive.

The functional roles represented in the benchmark suite include signal routing, instance routing, data-pipeline coordination, state management, hardware-job dispatch, timer or supervision logic, event-queue management, and arithmetic or DSP-oriented bitfield manipulation. The size of generated pseudo-C outputs also varies substantially, ranging from small single-purpose routines to larger multi-branch dispatch functions. This variation is important because the evaluation should test not only whether the pipeline handles simple load–compute–store cases, but also whether it remains faithful when control flow, predication, communication, and contextual information interact.

The benchmark suite does not cover every theoretically possible BFA program shape equally. Deeply nested loops are uncommon, because repeated execution in the studied codebase is often expressed through runtime re-invocation or state-transition patterns rather than through deeply nested local loops. Indirect calls and function pointers are also not central language features in this setting; dispatch is primarily represented through explicit message-passing or switch-like control flow. Similarly, complex heap-style data structures are not representative of the target routines, which mostly operate over flat context/state memory and architecture-specific registers. These absences are therefore partly properties of the target language and codebase rather than omissions of the benchmark-suite construction.

5.1.2 Representativeness

The benchmark suite is representative of the intended use case of this thesis: pseudo-C recovery for handwritten BFA routines used in real-time baseband firmware. It is drawn from production-oriented components and includes the main forms of evidence that the proposed pipeline is designed to preserve: local state updates, memory references, predicated operations, bundle-level parallelism, structured and unstructured control transfers, communication events, and domain-specific symbolic tokens.

The evaluated functions span several major component categories in the studied codebase. Table 5.2 summarises the main module groups represented in the benchmark suite. The distribution is not uniform: uplink and downlink subcarrier-processing components account for a large share of the suite. This reflects the practical concentration of BFA code in the studied firmware, but it also means that the benchmark suite is more representative of these real-time signal-processing components than of every possible baseband software layer.

Module category	**Functions**	**Representative domain**
Uplink subcarrier processing	163	FFT, beamforming, and stream coordination.
Downlink subcarrier processing	85	IFFT, beamforming, and time-domain pipeline coordination.
Downlink L1 digital beamforming	37	Weight calculation and data-indication handling.
Baseband control	18	Slot configuration and data-ready coordination.
Uplink L1 digital beamforming	15	Beam-server and PRACH-related handling.
Fronthaul interface	14	Packet handling and validation.
Compute scheduling	12	Thread synchronisation and job routing.
Beam data client	9	Response handling and counter management.
Real-time management	7	Timing, supervision, and recording-related routines.

Table 5.2: Main module categories represented in the benchmark suite.

The benchmark suite also includes both production routines and a smaller number of test or framework routines. The production routines represent the main target of the decompiler. Test and framework routines are retained because they exercise atypical instruction patterns and can reveal robustness issues. However, they are interpreted carefully in the outlier analysis, since their coding style and semantic context may differ from production routines.

Several limitations follow from this construction. First, the benchmark suite contains only instruction-containing routines. Placeholder artefacts without executable instructions are excluded because they cannot support any of the three evaluation axes. Second, the suite is dominated by the major signal-processing components where BFA is most heavily used, which may over-represent those domains relative to smaller components. Third, some test or framework routines may be simpler than production routines and could slightly inflate aggregate scores. These limitations are considered when interpreting the results in Chapter 6 and when discussing remaining failure modes in Chapter 7.

5.2 Axis 1: State Consistency

A faithful decompilation should preserve the type of each evaluated operation in the original BFA program. For example, a load should not be emitted as a store; an arithmetic update should not be collapsed into a plain assignment; and a bit-field extraction should not be rendered as an unrelated shift. Axis 1 quantifies this instruction-level operation fidelity by comparing the operation-type distribution

extracted from the ground-truth BFA evidence with the operation-type distribution extracted from the generated pseudo-C.

5.2.1 Operation-Type Taxonomy

We evaluated state consistency over a closed taxonomy of operation types. The taxonomy is defined by the semantic effect of an instruction rather than by its target-specific mnemonic. For example, a single `MOV` mnemonic may represent a load (when the source is a memory address), a store (when the destination is a memory address), or a plain assignment (when both operands are registers)—the classification depends on what the instruction *does*, not what it is *called*. This design avoids tying the metric to the surface syntax of a particular instruction set, makes the evaluation portable across architectures, and ensures that the framework can be described without disclosing proprietary instruction-set details.

Each evaluated instruction is assigned to exactly one operation type according to the rules in Table 5.3. The classification is derived from the normalised instruction record, including its operation class, operand roles, and operand types. Instructions that only serve as syntactic markers or do not correspond to an evaluated pseudo-C operation are excluded from this axis. Predicate markers, for example, are not counted as standalone operations when their conditions have already been propagated to the affected instructions.

Table 5.3: Sanitised operation-type taxonomy for state-consistency evaluation.

Type	**Classification rule**	**Illustrative example**
`load`	Read from memory into a scalar value	`x = mem[base + off]`
`store`	Write a scalar value to memory	`mem[base + off] = x`
`assign`	Copy or initialise a scalar value	`x = y`
`bitextract`	Extract, insert, test, set, or clear a bit-field (including single-bit operations)	`x = (y » k) & mask`
`arithmetic`	Integer arithmetic update	`x = y + z`
`compare`	Compare two values and update a condition	`cond = (x < y)`
`bitwise`	Bitwise logical operation	`x = y & mask`
`shift`	Logical or arithmetic shift	`x = y » k`
`branch`	Explicit control transfer or multi-way dispatch	`if (cond) goto L`
`send`	Message dispatch or resource management side-effect	`dispatch(portName)`
`other`	Evaluated operation not covered above	–

The taxonomy is intentionally expressed in terms of abstract operation effects. For example, a move-like instruction may be classified as a load, store, or assignment depending on whether its operands refer to memory or scalar values. Conversely, several target-specific instructions may be grouped into the same category when they represent the same kind of pseudo-C operation. The purpose of this axis is therefore

not to check whether the generated pseudo-C reproduces the original mnemonic, but whether it preserves the correct operation category.

5.2.2 Ground-Truth and Generated-Side Extraction

We extracted ground-truth operation labels from the CFG representation in the analysis JSON. Each instruction record provides the mnemonic, typed operands, and flow metadata needed to assign the instruction to one operation type in Table 5.3. The resulting label counts form the ground-truth operation distribution.

Generated-side extraction is performed after comment removal, since comments may contain operation-related terms that should not be counted as emitted code. The extractor first attempts to parse the pseudo-C using tree-sitter [Tree-sitter contributors, 2026] with a C grammar and classify statements by their AST node type. This follows the motivation of syntax-aware code evaluation, where surface text alone is insufficient for judging generated code Ren et al. [2020]. Assignments, compound expressions, conditions, memory accesses, function calls, and branch constructs are each mapped to the corresponding operation type in Table 5.3. If parsing fails or produces significantly fewer operations than expected, a conservative regex-based fallback extracts only high-confidence patterns, while ambiguous cases are mapped to `other`.

5.2.3 Axis 1 Scoring

Let $\mathcal{T}$ denote the operation-type taxonomy defined in Table 5.3. The scoring method follows the standard per-type precision, recall, and F1 formulation commonly used in information extraction and sequence-labelling evaluation [Doddington et al., 2004; Tjong Kim Sang and De Meulder, 2003]. For each operation type $t \in \mathcal{T}$, we compute a count-based F1 score. Let g_t denote the number of ground-truth operations of type t, and let p_t denote the number of generated pseudo-C operations classified as type t. Precision and recall are defined by comparing the overlap between the two counts:

$$\text{precision}_t = \frac{\min(p_t, g_t)}{\max(p_t, 1)}, \qquad \text{recall}_t = \frac{\min(p_t, g_t)}{\max(g_t, 1)}. \tag{5.1}$$

The per-type F1 score is then computed using the standard harmonic mean:

$$F_1^{(t)} = \begin{cases} \dfrac{2 \cdot \text{precision}_t \cdot \text{recall}_t}{\text{precision}_t + \text{recall}_t}, & \text{if precision}_t + \text{recall}_t > 0, \\ 0, & \text{otherwise.} \end{cases} \tag{5.2}$$

Let $\mathcal{T}_{gt}^{+}$ denote the set of operation types that appear in the ground truth:

$$\mathcal{T}_{gt}^{+} = \{t \in \mathcal{T} \mid g_t > 0\}. \tag{5.3}$$

The aggregated Axis 1 score is computed as a ground-truth-weighted mean over these operation types:

$$S_{\text{axis1}} = \frac{\sum_{t \in \mathcal{T}_{gt}^{+}} g_t \cdot F_1^{(t)}}{\sum_{t \in \mathcal{T}_{gt}^{+}} g_t}. \tag{5.4}$$

This formulation treats Axis 1 as a measure of how well the generated pseudo-C preserves the operation types present in the original BFA evidence. A missing

operation lowers recall for its type, while an extra operation lowers precision when it is classified into an operation type that also appears in the ground truth. The ground-truth weighting ensures that each original operation contributes proportionally to the final score. Operation types that appear only in the generated output are not included in the ground-truth-weighted aggregate; instead, their generated counts are retained as diagnostic evidence of over-generation.

5.3 Axis 2: Control-Flow Fidelity

Axis 1 measures whether the generated pseudo-C preserves the correct kinds of data-level operations, but it treats the program as an unordered distribution of operation types. It is therefore insensitive to when those operations execute, under which conditions they execute, and how externally visible events are ordered. Axis 2 evaluates this structural dimension by comparing the control-flow event skeleton of the ground-truth CFG with that of the generated pseudo-C.

Direct CFG topology comparison is not used. The ground-truth representation is a low-level CFG with explicit basic blocks, branches, and fall-through edges, whereas the generated pseudo-C may express the same behaviour using structured constructs such as `if`/`else`, `switch`, and loops. This kind of abstraction recovery is a standard objective of decompilation [Cifuentes, 1994; Schwartz et al., 2013]. A correct decompilation may therefore change the graph shape while preserving the program's structural behaviour. Axis 2 instead compares a projection of both representations onto a small event vocabulary.

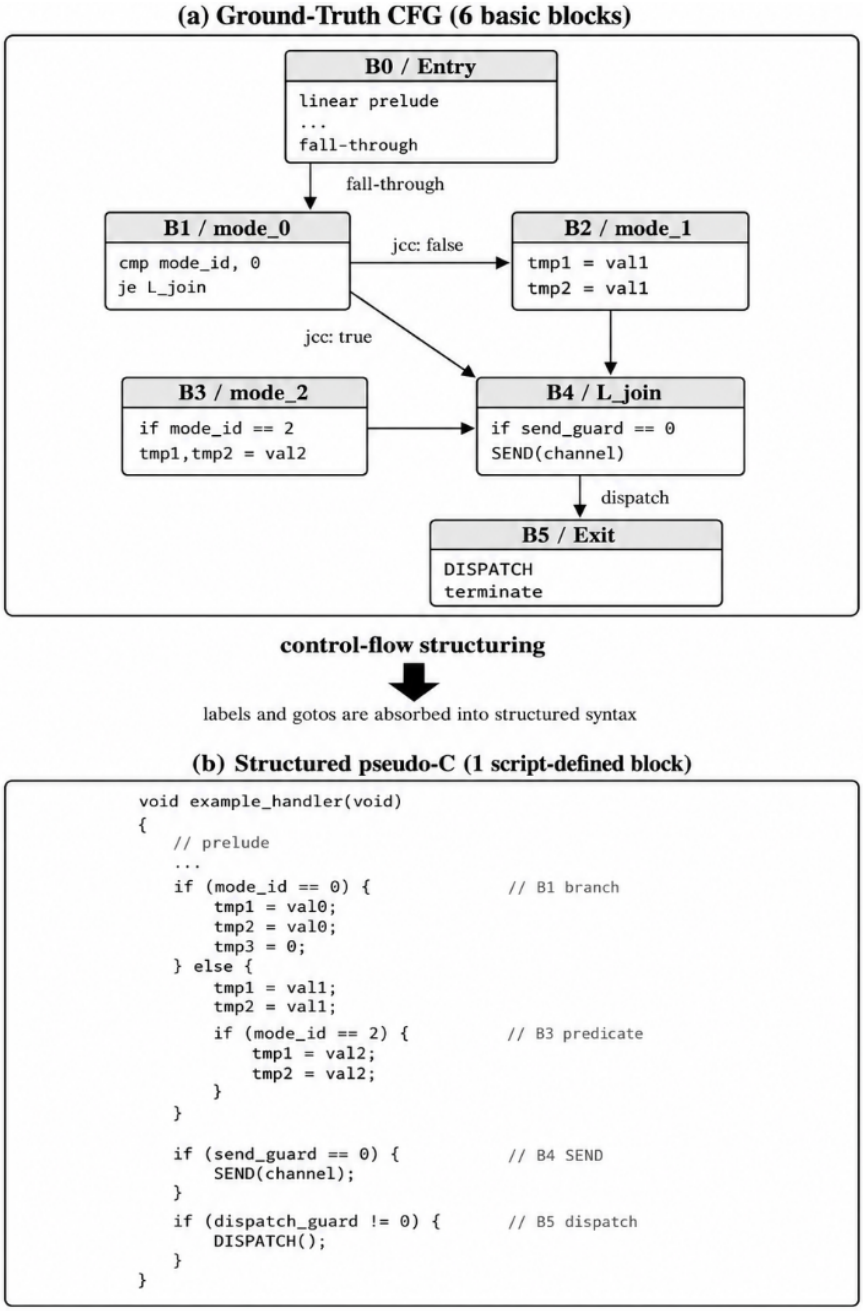


Figure 5.1: Abstraction gap between the ground-truth CFG and structured pseudo-C. The generated pseudo-C absorbs labels and jumps into structured constructs, so Axis 2 compares structural event traces rather than raw CFG topology.

Figure 5.1 illustrates why direct CFG-topology comparison is not suitable for Axis 2. The ground-truth side represents the program as a low-level CFG with explicit basic blocks, fall-through edges, conditional jumps, and exit transitions. The pseudo-C side represents the same behaviour after control-flow structuring. Instead of preserving each basic block as a separate labelled region, the decompiler rewrites the low-level edges into structured source-level constructs.

The conditional jump in B1 is absorbed into an `if/else` statement. The predicated update associated with B3 becomes a nested `if` condition inside the corresponding

branch. The guarded external event in B4 is represented as a conditional call, and the exit-side transition in B5 is represented as a guarded dispatch operation. As a result, labels, fall-through edges, and explicit jump targets disappear from the surface syntax, but their control-flow meaning is preserved through structured pseudo-C.

A topology-based metric would therefore penalise the generated pseudo-C for precisely the abstraction that decompilation is supposed to introduce. Axis 2 avoids this problem by comparing an abstract event trace instead of raw graph shape. In the figure, the conditional decision, predicated update, guarded external event, and dispatch operation are all preserved across the two representations, although their syntactic forms differ. This motivates the use of event-count and event-order fidelity rather than direct node- or edge-level CFG matching.

5.3.1 Structural Event Vocabulary

The event vocabulary contains three structural categories:

$$\mathcal{E} = \{\textsc{Send}, \textsc{Switch}, \textsc{Cond}\}. \tag{5.5}$$

Send represents externally visible communication or runtime events. Switch represents multi-way dispatch or state-transition points. Cond represents runtime conditional decisions that determine which execution path is taken. These categories are intentionally coarser than the full CFG. Their purpose is not to reconstruct every low-level edge, but to capture the behavioural skeleton that should remain visible after decompilation: where the program communicates externally, where it changes execution state, and under which conditions these events occur.

This event vocabulary also makes the metric comparable across abstraction levels. The ground-truth CFG and the generated pseudo-C may differ in the number of blocks and edges, but a faithful pseudo-C reconstruction should preserve the count and relative order of these structural events.

5.3.2 Ground-Truth Event Extraction

The ground-truth event sequence is extracted from the CFG representation in the analysis JSON. Blocks are traversed in program order, and each block is scanned for structural instructions and terminator metadata. Communication-like instructions emit a Send event, dispatch-like instructions emit a Switch event, and forward conditional terminators emit a Cond event.

Predicated structural operations require special handling because they combine conditional execution with an observable structural event. For example, a predicated communication instruction is both a guarded operation and an externally visible communication event. The predicate contributes a Cond event, while the communication instruction contributes a Send event. The extracted event order is therefore Cond followed by Send, reflecting that the guard condition is evaluated before the guarded action is taken.

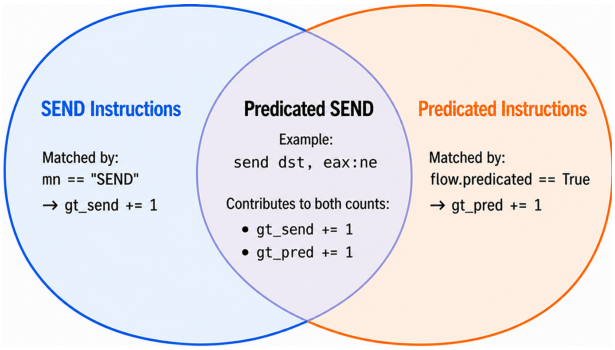


Figure 5.2: Counting rule for predicated structural operations in Axis 2. A predicated communication instruction contributes one COND event for its predicate and one SEND event for the communication operation.

Figure 5.2 illustrates this counting rule for a predicated communication instruction. Counting both parts is necessary because the guard and the communication event represent different structural facts: the former determines whether the operation executes, while the latter represents the externally visible action.

At the same time, the extractor avoids counting the same predicate repeatedly. When several consecutive structural operations share the same guard, they are treated as one guarded region and therefore contribute only one COND event. This matches the expected structured pseudo-C rendering, where the shared predicate is normally emitted as a single `if` block containing multiple guarded operations.

BFA bundle

```
MV a1, scratch.field1   :eax:ne
/* predicated on eax: write scratch.field1 */

MV a2, scratch.field2   :eax:ne
/* predicated on eax: write scratch.field2 */

SEND dst                :eax:ne
/* predicated on eax: send to dst */
```

Decompiled pseudo-C

```
if (eax != 0) {
    /* one shared if-block for predicate eax */
    scratch.field1 = a1;
    /* STORE a1 -> scratch.field1 */

    scratch.field2 = a2;
    /* STORE a2 -> scratch.field2 */

    SEND(dst);
    /* SEND dst */
}
```

Figure 5.3: Same-guard merging in Axis 2 event extraction. Multiple operations guarded by the same predicate correspond to one structured conditional region in the generated pseudo-C.

Figure 5.3 illustrates the same-guard merging rule. The shared guard contributes one COND event, while any guarded communication operation inside the region still contributes its own SEND event. This prevents a single predicate from being over-counted when it guards several adjacent operations.

Backward conditional branches are excluded from the COND count. Such branches usually represent loop-continuation edges rather than ordinary conditional divergence. In pseudo-C, they are expected to appear as structured loop constructs rather than as explicit `if` statements. Counting them as COND events on the ground-truth side would therefore introduce a systematic asymmetry and penalise correct loop reconstruction.

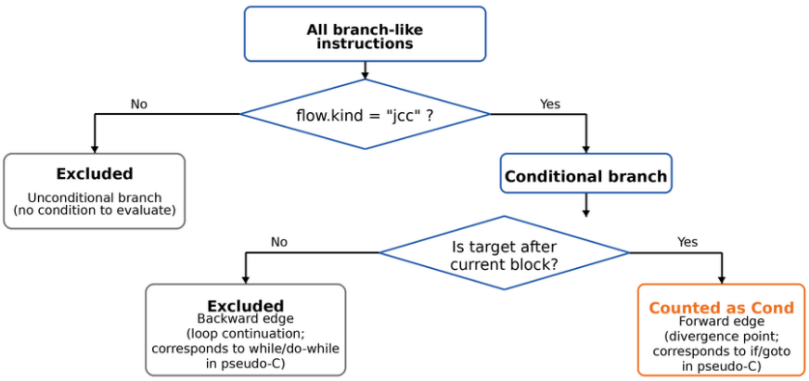


Figure 5.4: Filtering rule for extracting COND events from ground-truth branch instructions. Only conditional forward branches are counted as COND events.

Figure 5.4 summarises this filtering rule. Unconditional branches are excluded because they do not evaluate a runtime condition, while backward conditional branches are excluded because they usually represent loop-continuation edges that are expected to become structured loops in pseudo-C.

5.4 Axis 3: Semantic-Token Matching

The first two evaluation axes measure whether the generated pseudo-C preserves state-manipulation patterns (Axis 1) and control-flow structure (Axis 2). However, a structurally correct decompilation may still be misleading if it loses domain-specific identifiers such as memory field names, communication ports, hardware register fields, or symbolic constants. These identifiers give meaning to otherwise opaque loads, stores, messages, and bit-level operations.

Axis 3 therefore evaluates information fidelity: whether semantic tokens present in the ground-truth BFA instructions are preserved in the generated pseudo-C. This problem is related to variable name recovery Chen et al. [2022]; Lacomis et al. [2019] and type reconstruction Lee et al. [2011]; Noonan et al. [2016], but is more domain-specific. In BFA programs, field names describe data exchanged between pipeline stages, port names identify inter-component communication endpoints, and bit-field symbols document hardware layout constraints. Unlike lexical metrics such as BLEU Papineni et al. [2002] or CodeBLEU Ren et al. [2020], Axis 3 does not treat all tokens equally; it focuses only on identifiers whose absence would reduce the engineering value of the decompiled output.

5.4.1 Token Extraction

Token extraction projects both the ground-truth evidence and the generated pseudo-C onto a small vocabulary of domain-relevant identifiers. This avoids penalising harmless surface differences between low-level CFG instructions and higher-level pseudo-C, while still detecting the loss of names and symbols that carry program meaning.

We extract five categories of semantic tokens from the ground-truth CFG by traversing all instructions in each basic block. Four categories contribute to the aggregate score: *where* data resides (memory field names), *where* data is sent (message

port names), *how* data is sliced (bitfield parameters), and *what* data represents (symbolic constants). A fifth category—branch labels—is tracked as a diagnostic signal but excluded from scoring, as discussed below.

Memory field names. Memory field names capture the state variables that a function reads or writes. In DSP architectures designed for real-time signal processing, functions often communicate intermediate results through persistent memory regions, commonly represented as context or state buffers, rather than through register-passed arguments alone. The names of these fields are therefore not merely syntactic labels: they document the data-flow interface between pipeline stages.

Load and store instructions access such fields through memory-type operands, where the displacement field may encode the target field name. For example:

```
LD  r0, [base + #context.loopCounter]
ST  r1, [base + #context.loopCounter+4]
```

Both instructions reference the same logical field. After normalisation, the extracted token set therefore contains a single field-name token: `loopCounter`. If this token is absent from the generated pseudo-C, the output may still contain a load or store operation, but it no longer reveals which state variable is being accessed. This is a critical loss for downstream engineering tasks such as code review, debugging, and re-implementation.

Message port names. Message port names identify the communication endpoints used by a function. The architecture's `SEND` primitive dispatches inter-component messages to named destination ports, so the first operand of each `SEND` instruction is treated as a semantic token. For example:

```
SEND  stageConfigDonePort, r2
```

yields the token `stageConfigDonePort`. Preserving this token is important because the generated pseudo-C should reveal not only that a message is sent, but also which interface or pipeline stage receives it. If the port name is omitted or replaced by a generic call target, the output loses the communication topology that explains the function's role in the wider system.

Bitfield parameters. Bitfield parameters capture layout information encoded in bit-level operations. In embedded and DSP-oriented programs, packed registers, status words, and message payloads often store multiple logical fields within a single machine word. The constants used to extract or insert these fields are therefore not incidental numeric values; they specify the position and size of hardware- or protocol-defined fields.

We normalise each bitfield access into an $(\text{offset}, \text{mask})$ token, where offset identifies the starting bit position and mask delimits the field width. In the instruction encoding, the width operand w specifies the index of the highest bit within the extracted field (i.e. the field spans $w + 1$ bits). The mask is therefore computed as:

$$\text{mask} = (1 \ll (w + 1)) - 1. \tag{5.6}$$

For example, an extraction with offset 5 and width operand 3 denotes a four-bit field starting at bit 5, producing the token $(5,\ \texttt{0xF})$, since $(1 \ll 4) - 1 = \texttt{0xF}$. In generated

pseudo-C, the same access may appear as `(x » 5) & 0xF`. During matching, the mask is checked in both hexadecimal and decimal representations to accommodate surface-level formatting differences in the generated code. Preserving this token is important because changing either the offset or the mask changes which logical field is read or written, even if the surrounding expression remains syntactically similar.

Symbolic constants. Immediate operands fall into two sub-categories of interest:

- *Named symbols*: operands flagged with `isSymbol=true` whose names exceed four characters and begin with an uppercase letter (e.g., `MAX_CARRIERS`). These represent compile-time configuration constants.
- *Large numeric literals*: unsigned values exceeding 255, which typically encode address offsets, buffer sizes, or protocol-mandated magic numbers (e.g., `4096` for FFT size, `0x3FFF` for a 14-bit mask).

Small immediates (≤ 255) are excluded as they frequently represent generic loop bounds, register indices, or alignment padding that carry minimal semantic weight.

Branch labels (diagnostic only). Conditional and unconditional branch instructions reference named label targets that encode the programmer's intent for control-flow decisions. For example:

```
BEQ   #handleErrorPath
JMP   #loopComplete
```

yields tokens `handleErrorPath` and `loopComplete`. Branch labels are extracted and tracked but *excluded from the aggregate Axis 3 score*. The reason is that a well-performing decompiler often lifts low-level branch-and-label patterns into structured control flow (`if`/`while`/`do-while`), causing the original label names to disappear. In such cases, a low label recall reflects successful abstraction rather than information loss. We retain the per-category recall as a diagnostic signal and discuss its implications in Section 6.2.3.

5.4.2 Token Matching and Scoring

Once semantic tokens are extracted from the ground-truth instructions, each token is matched against the generated pseudo-C to determine whether it has been preserved. Matching is order-independent: each ground-truth token is searched for anywhere in the generated output, since structural ordering is already assessed by Axis 2 (Section 5.3).

Each category employs a matching strategy tailored to how the token is likely to surface in higher-level code. Textual tokens (memory field names, symbolic names) are matched via case-insensitive substring search with underscore normalisation to tolerate minor formatting differences. Structural tokens (message port names) require the name to appear within the expected call pattern. Numeric tokens (bitfield parameters, large literals) are matched in both hexadecimal and decimal representations; for bitfield pairs, partial credit is awarded when the offset is present but the mask is absent or expressed in an unrecognised form. Branch labels, while tracked using the same substring matching, do not contribute to the aggregate score.

For each scored category $c \in C_{\text{scored}}$, we compute a per-category recall:

$$R_c = \frac{\text{matched}_c}{\text{total}_c} \tag{5.7}$$

where total_c is the number of ground-truth tokens in category c and matched_c is the (possibly fractional) count of successfully matched tokens. Categories with no ground-truth tokens are assigned a neutral recall of 1.0 so that they do not penalise the overall score.

The aggregate Axis 3 score is the weighted mean recall across the four scored categories:

$$\text{Axis3} = \frac{\sum_{c \in C_{\text{scored}}} \text{total}_c \cdot R_c}{\sum_{c \in C_{\text{scored}}} \text{total}_c} \tag{5.8}$$

where $C_{\text{scored}} = \{\text{memory fields},\ \text{message ports},\ \text{bitfield parameters},\ \text{symbolic constants}\}$. This weighting ensures that categories containing more tokens contribute proportionally to the final score, reflecting the intuition that a function rich in bitfield accesses should be penalised more heavily for missing parameters than a function containing only a single such access.

Chapter 6

Results

This chapter presents the evaluation results of the LLM-based decompiler across 527 BFA functions using the three-axis framework defined in Chapter 5.

6.1 Overall Results

Table 6.1 summarises the mean scores across all three evaluation axes, grouped by instruction-count tier. We partition the dataset into three instruction-count tiers based on the number of evaluated instructions in the original BFA function: *small* ($\leq$ 7 instructions, 318 files), *medium* (8–18 instructions, 126 files), and *large* ($>$ 18 instructions, 83 files). These thresholds were chosen to reflect natural breakpoints in function complexity rather than to produce equally sized groups: small functions typically perform a single load–compute–store sequence, medium functions contain one or two conditional branches with multiple side effects, and large functions involve complex multi-path control flow with numerous predicated operations and multiple communication events.

Across the full benchmark suite, the pipeline achieves mean scores of 0.971 for Axis 1 state consistency, 0.975 for Axis 2 control-flow fidelity, and 0.989 for Axis 3 semantic-token preservation. To make these aggregate results more informative, Table 6.2 reports the corresponding score distribution, including standard deviation, minimum, and maximum values observed in the evaluation run. This clarifies that the headline scores are dataset-level means across the 527 evaluated BFA functions rather than scores from a single example.

All three axes achieve mean scores above 0.97, indicating that the decompiler preserves operation types, control-flow structure, and semantic identifiers with high fidelity across the dataset. However, these averages should not be interpreted as uniform performance across all functions. The standard deviations and minimum scores show that a small number of outlier functions still reduce per-function fidelity. This is expected in an LLM-assisted pipeline, where the final pseudo-C rendering stage is not fully deterministic across runs. Therefore, the aggregate scores are interpreted together with the per-axis distribution, outlier cases, and function-size effects in the following subsections.

Figure 6.1 shows pairwise scatter plots of the three axes. The outliers do not strongly cluster across axes, which indicates that the three metrics capture complementary failure modes: a function with reduced operation-category preservation does not necessarily have reduced control-flow fidelity or semantic-token preservation. This supports the use of the three-axis evaluation framework as a diagnostic

instruction-count	n	Axis 1	Axis 2	Axis 3
Small ($\leq$ 7 instr.)	318	0.987	0.998	0.991
Medium (8–18 instr.)	126	0.959	0.968	0.987
Large ($>$ 18 instr.)	83	0.926	0.894	0.986
All	527	0.971	0.975	0.989

Table 6.1: Mean evaluation scores by function instruction-count across 527 BFA functions.

Axis	Mean	Std. dev.	Min	Max
Axis 1: State consistency	0.971	0.060	0.667	1.000
Axis 2: CFG fidelity	0.975	0.094	0.417	1.000
Axis 3: Semantic-token preservation	0.989	0.035	0.500	1.000

Table 6.2: Distribution of evaluation scores across 527 BFA functions.

tool rather than a single surface-level similarity score.

6.2 Per-Axis Detailed Analysis

6.2.1 Axis 1: State Consistency

Figure 6.2 shows the GT-weighted per-type F1 score broken down by instruction-count group. The strongest categories are `store` (0.99), `send` (1.00), `load` (0.97), and `assign` (0.96), indicating that the LLM reliably preserves memory accesses, message dispatches, and scalar data movement. `bitextract` (0.93) and `shift` (0.96) also perform well, confirming that bit-level operations are faithfully rendered.

Type	F1	GT count	Gen count	Gen/GT
compare	0.83	303	474	1.56
bitwise	0.82	113	107	0.95
branch	0.73	277	193	0.70

Table 6.3: Axis 1 categories with systematically lower F1 scores.

Table 6.3 summarises the three categories with systematically lower F1 scores. Their errors are not uniform: `compare` is mainly over-generated, `branch` is under-generated, while `bitwise` shows a smaller count difference but lower label agreement. The underlying causes differ by category:

- `compare`: Over-generation (56%) stems from two sources—the LLM adding defensive guards before suspend/dispatch operations, and a classification artefact where GT bit-test instructions (counted as `bitextract`) are expressed as `if ((x » k) & 1)` in the output, which the generated-side classifier counts as `compare`.
- `bitwise`: Under-generation occurs when standalone mask operations are absorbed into compound bit-field extractions. For example, `x &= 0xFF` may be merged into `(y » 8) & 0xFF`, causing reclassification as `bitextract`.

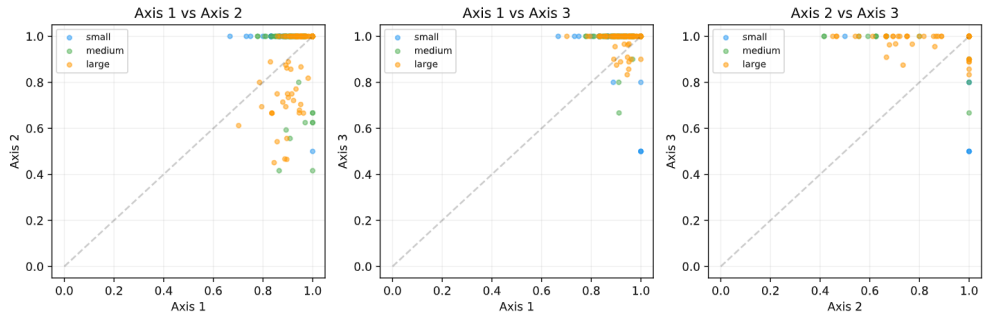


Figure 6.1: Pairwise scatter plots of the three evaluation axes. Each point represents one BFA function, coloured by instruction-count group. Outliers on one axis do not cluster on another, confirming that the three axes capture independent failure modes.

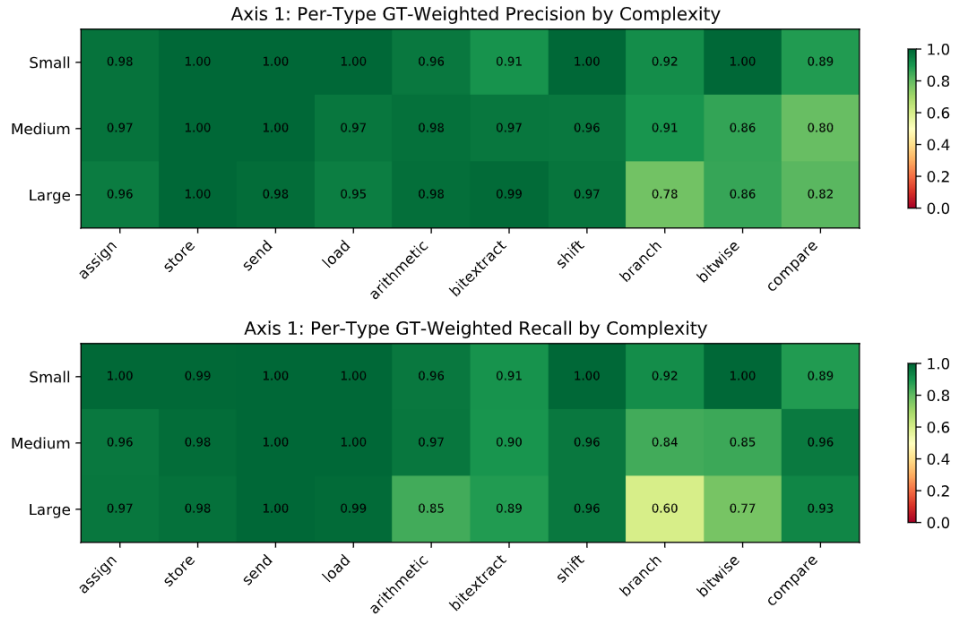


Figure 6.2: GT-weighted per-type F1 scores for Axis 1 (State Consistency), grouped by function instruction-count. Darker cells indicate lower scores.

- `branch`: Under-generation (Gen/GT = 0.70) results from the LLM lifting conditional jumps into `if/else` (counted as `compare`), back-edges into `while`, and exit jumps into `return`—all semantically correct but invisible to the branch counter.

In all, the lower scores in these three categories do not indicate semantic errors in the generated code. Rather, they arise because the ground-truth classifier and the generated-side classifier operate at different abstraction levels. The ground-truth classifier assigns one type per assembly instruction based on its mnemonic; the generated-side classifier assigns types based on C syntax structure. When the LLM produces idiomatic C—writing `if ((x » k) & 1)` instead of a separate bit-test, folding a mask into a shift expression, or replacing a jump with structured control flow—the same operation receives different type labels on each side. This is an inherent limitation of count-based metrics that compare code across abstraction levels.

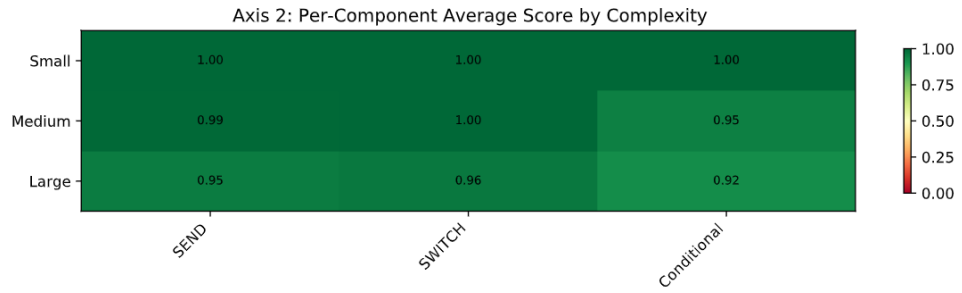


Figure 6.3: Axis 2 control-flow fidelity scores grouped by instruction-count tier. Darker cells indicate lower scores.

Table 6.4: Axis 2 mean score by instruction-count bin.

Instruction range	n	Axis 2 mean
1–7	110	1.000
8–18	126	0.968
19–40	51	0.910
41–80	18	0.888
81–200	14	0.848

Conclusion for RQ1. The generated pseudo-C preserves instruction-level operation categories to a high extent, with a mean Axis 1 state-consistency score of 0.971 across 527 functions, a standard deviation of 0.060, and a range from 0.667 to 1.000. The remaining reductions are concentrated in compare, bitwise, and branch categories, and mainly reflect abstraction-level mismatches and expression merging rather than systematic loss of state-changing operations.

6.2.2 Axis 2: CFG Fidelity

or Figure 6.3 shows the per-component scores for Axis 2. Message dispatch events achieve near-perfect fidelity across all instruction-count groups (0.99–1.00), and multi-way suspend points are equally well preserved (0.99–1.00). The main source of degradation is the *conditional* component, which measures whether the generated pseudo-C contains the correct number of condition-guarded operations.

A first explanation for this degradation is function instruction-count. Table 6.4 shows that the Axis 2 score decreases monotonically with instruction count, dropping from 1.000 for the smallest functions to 0.848 for functions exceeding 80 instructions. Table **??** includes only functions for which the ground-truth extractor identifies at least one Axis 2 structural event, namely a Cond, Send, or Switch event.

However, instruction-count alone does not explain the lowest-scoring cases. A closer inspection of the 24 files scoring below 0.7 reveals a stronger factor: 22 of them execute at a non-entry execution point, such as resuming after an asynchronous response or handling an externally supplied signal, where the input data layout is ambiguous. This corresponds to the context-extraction limitation described in Section 7.2. In these cases, the pipeline cannot always determine which data-layout struct applies at the current execution point, so the LLM receives multiple candidate definitions without disambiguation.

This observation suggests that Axis 2 degradation has two layers. At the aggreg-

ate level, larger functions are more difficult because they contain more conditionals and externally visible events. At the outlier level, the dominant cause is not function size itself, but incomplete input context. When the context pipeline provides unambiguous data-layout definitions, the LLM preserves the control-flow event structure with high fidelity even for large functions.

A manual review of the Axis 2 outliers identifies four failure modes:

- *Conditional over-generation* (9 files): The LLM inserts additional `if` guards not present in the ground truth. This occurs when the input context provides multiple candidate data-layout definitions without indicating which one applies at the current execution point; the LLM generates a defensive check for each candidate interpretation before performing a dispatch or store. In 2 of these files, the extra guard also causes a dispatch call to be duplicated inside each branch.
- *Conditional under-generation* (9 files): The LLM omits a condition that the ground truth evaluates. Seven of the 9 files are hardware-variant copies of the same base function, where the ground truth contains a predicated operation whose condition is set earlier in the control flow; the LLM merges this predicated path into an unconditional sequence, dropping the guard.
- *Extra dispatch* (3 files): The LLM generates one additional message-dispatch call. All three are signal-routing functions where the ground truth dispatches once, but the LLM interprets an ambiguous operand as a second dispatch target.
- *Event ordering mismatch* (3 files): Event counts match the ground truth exactly, but the generated ordering differs. The most prominent case involves 12 conditional checks followed by 12 dispatches; the LLM instead interleaves each check with its corresponding dispatch. The remaining 2 files swap the relative order of a suspend point and its preceding condition. These are semantically valid reorderings that the sequence-comparison metric penalises.

This manual analysis shows that low Axis 2 scores should not be interpreted uniformly. Conditional over-generation, conditional under-generation, and extra dispatches are genuine output defects because they change the recovered control-flow event structure. These account for 21 of the 24 outliers, and they primarily trace back to ambiguous or incomplete input context rather than to a general inability of the LLM to express structured control flow. By contrast, the 3 event-ordering mismatches are false negatives of the evaluation metric: the generated code performs the same checks and dispatches the same messages, but arranges them in a different order that the LCS-based sequence comparison penalises.

Conclusion for RQ2. The generated pseudo-C preserves the control-flow event skeleton to a high extent, with a mean Axis 2 control-flow-fidelity score of 0.975 across 527 functions, a standard deviation of 0.094, and a range from 0.417 to 1.000. This indicates that conditional decisions, switch-like dispatches, and communication events are usually preserved when the supplied evidence is unambiguous, while the lowest scores are mainly caused by ambiguous context extraction and, in a smaller number of cases, by ordering artefacts in the sequence-based metric.

6.2.3 Axis 3: Semantic Token Matching

Figure 6.4 shows the per-category recall across instruction-count groups. Message port names achieve perfect recall (1.000) in all groups—the LLM never drops a com-

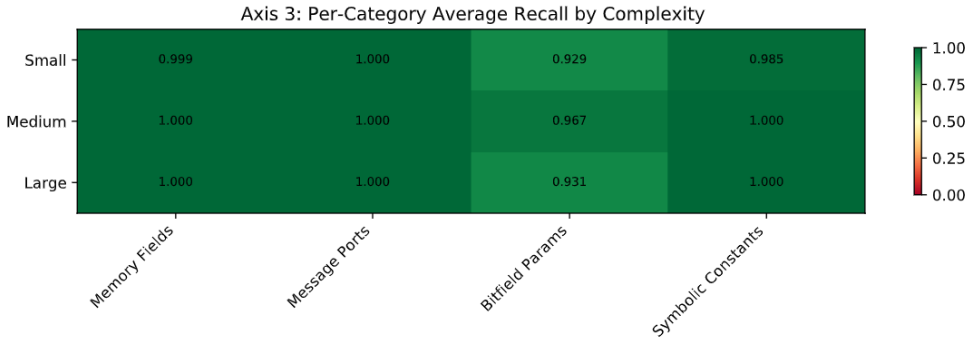


Figure 6.4: Axis 3 per-category recall by instruction-count group. Only the four scored categories are shown; branch labels (diagnostic only) are excluded from the aggregate.

munication endpoint identifier. Memory field names (0.996) and symbolic constants (0.980) are similarly strong, confirming that the LLM preserves the domain-specific vocabulary that gives meaning to data accesses and configuration values.

Bitfield parameters exhibit the lowest scored recall (0.944) among the four categories, primarily due to partial-credit cases where the shift offset is correctly reproduced but the mask literal appears in a form not recognised by the matcher. For example, a ground-truth token $(5,$ `0xFFF8`$)$ expects the generated code to contain `» 5` and `& 0xFFF8`. If the LLM instead writes `tokenVal &= 0xFFF80000`, the mask value is present but embedded within a larger literal, causing the pattern match to fail. In such cases the offset receives full credit while the mask does not, yielding a partial score of 0.5 rather than 1.0. This represents a matcher coverage limitation rather than an LLM error—the correct value is present in the output but expressed in a form the evaluation regex does not cover.

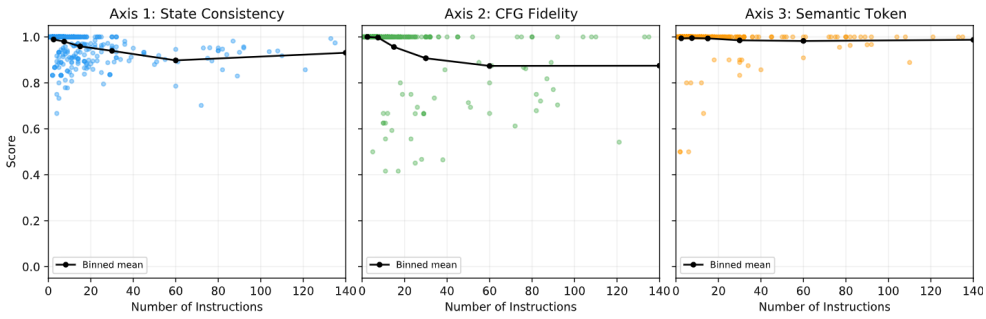


Figure 6.5: Score versus instruction count for each axis. Each point represents one BFA function. The black line shows the binned mean. Axis 3 remains flat regardless of function size, while Axis 2 exhibits the steepest degradation.

As shown in the right panel of Figure 6.5, unlike the first two axes, Axis 3 remains nearly flat across all function sizes, with no statistically meaningful degradation even for the largest functions. This indicates that semantic-token preservation is independent of function instruction-count—the LLM retains domain-specific identifiers regardless of how many instructions surround them.

Aside from the categories shown in the heatmap, branch labels are also tracked as a diagnostic signal but excluded from the aggregate score. They show a recall of 0.230 across 65 files. Manual inspection confirms that the low recall reflects successful

abstraction rather than information loss: the LLM lifts branch-and-label patterns into structured `if`/`while`/`do-while` constructs, causing the original label names to disappear from the output. In the minority of cases where the control flow is too complex for structured lifting, the LLM retains explicit `goto` statements with the original label names.

Three of the four Axis 3 outliers involve multi-digit hexadecimal constants where the LLM produces an incorrect digit. The remaining outlier loses one bitfield parameter when the mask is absorbed into a compound expression. All four are test-component functions with atypical instruction patterns not representative of production code. Unlike the Axis 1 and Axis 2 outliers, these are genuine LLM errors—the expected value is absent from the output, not merely expressed in an unrecognised form—but their restriction to test components suggests they reflect edge-case numeric reasoning failures rather than a systematic limitation.

Conclusion for RQ3. The generated pseudo-C preserves concrete semantic tokens very strongly, with a mean Axis 3 semantic-token-preservation score of 0.989 across 527 functions, a standard deviation of 0.035, and a range from 0.500 to 1.000. This is the most stable of the three axes: message-port names are preserved perfectly, memory-field names and symbolic constants are preserved with near-perfect recall, and the remaining losses are limited mainly to bitfield-mask formatting issues and rare incorrect hexadecimal literals.

Chapter 7

Discussion and Conclusion

The results in Chapter 6 show that the proposed pipeline achieves high fidelity on most functions, but the remaining errors are not caused by a single factor. This chapter separates three sources of limitation—evaluation metrics, context extraction, and LLM capability—and concludes with a summary of contributions and future directions.

7.1 Evaluation Metric Limitations

As discussed in Section 2.2.1, decompiler correctness cannot be assessed through syntactic correspondence alone [Liu and Wang, 2020], and useful evaluation must address multiple dimensions including fidelity, consistency, and readability [Cao et al., 2024; Dramko et al., 2024]. The three-axis framework used in this thesis operationalises this principle, but each axis remains an approximation. Axis 1's mnemonic-based GT classifier diverges from the AST-based generated-side classifier when the LLM performs valid abstraction lifting (e.g., branch → `if/else`). Axis 2's sequence metric penalises semantically equivalent reorderings (3 of 24 outliers). Axis 3's regex matcher misses values embedded in compound expressions. A more robust future evaluation could incorporate SMT-based equivalence checking [Lopes et al., 2021] or human-centred assessment of practical interpretability [Burk et al., 2022; Eom et al., 2024].

7.2 Context Pipeline Limitations

The dominant source of real output degradation is incomplete input context. We identify three categories of context-pipeline limitations:

1. *Ambiguous data-layout resolution* (5.1% of files): Functions executing at non-entry points receive data whose layout is defined by an external component. The pipeline collects all candidate definitions but cannot disambiguate which applies at the current execution point, causing the LLM to default to the entry-level definition. This drives the majority of Axis 2 outliers. The challenge is analogous to the type-recovery problem in binary analysis [Lee et al., 2011], where indirect data flows obscure the concrete type at a given program point.

2. *Missing definitions in test components* (2.1%): Test functions manipulate registers directly without defining layout abstractions. The LLM falls back to raw register names—instruction-faithful but lacking semantic field names.

3. *Incomplete caller resolution* (14.6% of production files): Functions registered through imperative runtime calls are invisible to the declarative-file search. However, 42 of 52 affected functions are stateless signal-routing handlers, so the missing caller affects only documentation comments rather than instruction-level correctness.

These limitations are structural rather than fundamental. A promising direction is to apply retrieval-augmented generation (RAG) [Lewis et al., 2020] to the context discovery problem: rather than relying solely on deterministic path-based search, a RAG-based system could index all interface definitions across the codebase and retrieve the most relevant data-layout struct for a given execution point based on semantic similarity to the function's register-access pattern. This would address category (1) without requiring full inter-component static analysis, and could also improve category (3) by retrieving caller information from C source files that the current declarative search does not parse.

7.3 LLM Capability Boundaries

With complete context, the LLM achieves near-perfect scores for functions under 18 instructions (0.987/0.998/0.995). Two genuine capability limits remain:

- *instruction-count scaling*: Axis 2 degrades from 1.000 to 0.848 as function size exceeds 80 instructions, consistent with long-context reasoning degradation in transformer models [Liu et al., 2024]. Axis 3 remains flat, confirming that semantic-token recall is instruction-count-independent.
- *Numeric reasoning*: The four Axis 3 outliers involve multi-digit hexadecimal constants where the LLM produces an incorrect digit, consistent with known transformer limitations in precise arithmetic [Dziri et al., 2023].

The 9 conditional-over-generation cases represent a context–capability interaction: the model receives ambiguous input and hedges by inserting guards for multiple interpretations, paralleling "hallucination under uncertainty" in code generation [Chen et al., 2021]. Improving context disambiguation—whether through enhanced static analysis or RAG-based retrieval—would likely eliminate these without requiring changes to the model.

7.4 Error Attribution Summary

The preceding sections analysed limitations from three perspectives—metric design, context completeness, and model capability. To provide a unified view, Table 7.1 consolidates all score reductions across the 527 evaluated functions and assigns each to a primary root cause.

Three observations emerge from this attribution. First, the single largest category—expression merging (135 files)—is a grey area: the LLM combines adjacent operations into compound C expressions (e.g., merging a load and a shift into a single subscript access), which improves readability but causes the per-type counter to register fewer discrete operations than the ground truth. Second, the 29 context-gap files are the only category where the output is unambiguously wrong in a way that affects developer comprehension; these are directly addressable through the pipeline improvements discussed in Section 7.2. Third, only 3 files exhibit a pure LLM reasoning error

Table 7.1: Attribution of score reductions across all 527 BFA functions. Files may appear in multiple rows where issues co-occur.

Root cause	**Axis**	**Files**	**Nature**
Abstraction lifting: branch → `if/else`	1	109	Evaluation Metric artefact
Expression merging: operations absorbed into compound expressions	1	135	Mixed[a]
Ambiguous data-layout context	2	29	Context gap
Semantically equivalent event reordering	2	12	Evaluation Metric artefact
Bitfield mask form not recognised by matcher	3	17	Evaluation Metric artefact
Incorrect numeric constant (multi-digit hex)	3	3	LLM error
Perfect on all three axes			360 (68.3%)
Imperfect on at least one axis[b]			167 (31.7%)

[a] Partly evaluation-metric artefact (expression-boundary ambiguity), partly genuine LLM imprecision (over-merging adjacent operations).
[b] At least one of the three axis scores is below 1.0.

(incorrect hex digit), confirming that the model's generation capability is not the practical bottleneck.

These findings motivate the priorities outlined in the following section: context disambiguation offers the highest return on investment, followed by metric refinement, with model-level improvements being the least urgent.

7.5 Summary and Future Work

This thesis presented a hybrid decompilation pipeline for handwritten BFA routines, combining deterministic static analysis with LLM-based pseudo-C generation. Evaluated on 527 functions, the system achieves aggregate scores of 0.971 for state consistency, 0.975 for control-flow fidelity, and 0.989 for semantic-token preservation. These results show that an evidence-constrained LLM can act as an effective pseudo-C rendering backend when behaviourally important program facts are first recovered by deterministic analysis.

The central finding is that output quality is bounded less by the model's general code-generation ability than by the completeness and disambiguation of the evidence supplied to it. Most remaining failures are caused by incomplete context or by evaluation artefacts introduced when low-level control flow is validly lifted into higher-level pseudo-C. Pure LLM capability errors are rare and mainly involve precise numeric literals. Future work should therefore focus on how AI-assisted decompilation should represent, retrieve, validate, and evaluate program evidence.

Context-aware decompilation under incomplete evidence. One open research direction is how to recover and rank contextual evidence for low-level routines whose meaning depends on surrounding codebase artefacts. The present pipeline uses deterministic lookup rules to recover data-layout definitions, caller metadata, and execution-point information. The results show that ambiguous context can lead the LLM to generate extra guards, duplicate dispatches, or choose an incorrect inter-

pretation. Future work could therefore study retrieval-augmented context recovery, where relevant definitions are selected not only by textual similarity, but also by compatibility with the routine's register use, memory accesses, control-flow position, and communication behaviour [Lewis et al., 2020].

Semantics-aware evaluation and verification. A second direction concerns how to evaluate and verify pseudo-C that is more abstract than the original low-level representation. The three-axis framework used in this thesis avoids surface-level text similarity, but it can still penalise valid abstraction lifting, such as replacing branches with structured `if/else` constructs or merging bit operations into compound expressions. Future work could combine CFG-level evidence, event traces, and expression-level equivalence checking to distinguish real semantic errors from metric artefacts. A related direction is post-generation verification: generated literals, operation directions, predicate scopes, and communication events could be checked against the deterministic skeleton before the pseudo-C output is accepted [Lopes et al., 2021].

Generalisation beyond BFA. Although this thesis focuses on BFA, the underlying problem is broader. Many industrial systems contain low-level, domain-specific, or hardware-control languages for which no standard decompiler, aligned training corpus, or public benchmark exists. Future work could investigate whether the same evidence-constrained design transfers to other proprietary assembly languages, DSP control languages, packet-processing languages, or hardware configuration DSLs. This would test whether the main idea is BFA-specific, or whether it offers a more general pattern for applying LLMs in low-resource decompilation: recover mechanical evidence deterministically, expose uncertainty explicitly, and use the model only for readable rendering [Armengol-Estapé et al., 2024; Tan et al., 2024].

Human-centred evaluation of pseudo-C usefulness. Finally, this thesis evaluates fidelity to recovered program evidence, but the practical motivation of decompilation is program comprehension. A natural next step is a human-centred study with engineers who need to inspect, debug, or review low-level routines. Such a study could ask whether generated pseudo-C reduces comprehension time, improves bug localisation, increases confidence in code reviews, or helps engineers navigate unfamiliar components. This is important because an output can be faithful according to structural metrics while still being difficult to use, or imperfect according to metrics while still being practically helpful [Burk et al., 2022; Eom et al., 2024].

Bibliography

Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Pearson/Addison-Wesley, 2 edition, 2007.

Frances E. Allen. Control flow analysis. *SIGPLAN Notices*, 5(7):1–19, July 1970. doi: 10.1145/390013.808479. URL https://doi.org/10.1145/390013.808479.

ARM Limited. *ARM Architecture Reference Manual: ARMv7-A and ARMv7-R Edition*. ARM Limited, 2012. URL https://developer.arm.com/documentation/ddi0406/c/. Issue C, Document ID DDI0406C.

Jordi Armengol-Estapé, Jackson Woodruff, Alexander Brauckmann, José Wesley de Souza Magalhães, and Michael F. P. O'Boyle. ExeBench: An ML-scale dataset of executable C functions. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, MAPS 2022, pages 50–59, New York, NY, USA, 2022. Association for Computing Machinery. doi: 10.1145/3520312.3534867. URL https://doi.org/10.1145/3520312.3534867.

Jordi Armengol-Estapé, Jackson Woodruff, Chris Cummins, and Michael F. P. O'Boyle. SLaDe: A portable small language model decompiler for optimized assembly. In *2024 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 67–80. IEEE, 2024. doi: 10.1109/CGO57630.2024.10444788. URL https://doi.org/10.1109/CGO57630.2024.10444788.

Zion Leonahenahe Basque, Ati Priya Bajaj, Wil Gibbs, Jude O'Kain, Derron Miao, Tiffany Bao, Adam Doupé, Yan Shoshitaishvili, and Ruoyu Wang. Ahoy SAILR! there is no need to DREAM of C: A compiler-aware structuring algorithm for binary decompilation. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 361–378, Philadelphia, PA, USA, August 2024. USENIX Association. ISBN 978-1-939133-44-1. URL https://www.usenix.org/conference/usenixsecurity24/presentation/basque.

Tim Bray. The JavaScript object notation (JSON) data interchange format, December 2017. URL https://www.rfc-editor.org/info/rfc8259.

Kevin Burk, Fabio Pagani, Christopher Kruegel, and Giovanni Vigna. Decomperson: How humans decompile and what we can learn from it. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 2765–2782, Boston, MA, USA, August 2022. USENIX Association. URL https://www.usenix.org/system/files/sec22-burk.pdf.

Ying Cao, Ruigang Liang, Kai Chen, and Peiwei Hu. Boosting neural networks to decompile optimized binaries. In *Proceedings of the 38th Annual Computer Security Applications Conference*, ACSAC '22, pages 508–518, New York, NY, USA,

2022. Association for Computing Machinery. doi: 10.1145/3564625.3567998. URL https://doi.org/10.1145/3564625.3567998.

Ying Cao, Runze Zhang, Ruigang Liang, and Kai Chen. Evaluating the effectiveness of decompilers. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA '24, pages 491–502, New York, NY, USA, 2024. Association for Computing Machinery. doi: 10.1145/3650212.3652144. URL https://doi.org/10.1145/3650212.3652144.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. doi: 10.48550/arXiv.2107.03374. URL https://arxiv.org/abs/2107.03374.

Qibin Chen, Jeremy Lacomis, Edward J. Schwartz, Claire Le Goues, Graham Neubig, and Bogdan Vasilescu. Augmenting decompiler output with learned variable names and types. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 4327–4343, Boston, MA, USA, August 2022. USENIX Association. ISBN 978-1-939133-31-1. doi: 10.48550/arXiv.2108.06363. URL https://www.usenix.org/conference/usenixsecurity22/presentation/chen-qibin.

Cristina Cifuentes. *Reverse Compilation Techniques*. PhD thesis, Queensland University of Technology, Brisbane, Australia, 1994. URL https://www.cs.tufts.edu/~nr/cs257/archive/cristina-cifuentes/DCC_decompilation_thesis.pdf.

Cristina Cifuentes and K. John Gough. Decompilation of binary programs. *Software: Practice and Experience*, 25(7):811–829, July 1995. doi: 10.1002/spe.4380250706. URL https://doi.org/10.1002/spe.4380250706. Publisher: John Wiley & Sons, Ltd.

Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. Efficiently computing static single assignment form and the control dependence graph. *ACM Transactions on Programming Languages and Systems*, 13 (4):451–490, 1991. doi: 10.1145/115372.115320. URL https://doi.org/10.1145/115372.115320.

George Doddington, Alexis Mitchell, Mark Przybocki, Lance Ramshaw, Stephanie Strassel, and Ralph Weischedel. The automatic content extraction (ACE) program: Tasks, data, and evaluation. In *Proceedings of the Fourth International Conference on Language Resources and Evaluation (LREC 2004)*, Lisbon, Portugal, 2004. European Language Resources Association. URL https://aclanthology.org/L04-1011/.

Luke Dramko, Jeremy Lacomis, Edward J. Schwartz, Bogdan Vasilescu, and Claire Le Goues. A taxonomy of C decompiler fidelity issues. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 3147–3164, Philadelphia, PA, USA,

August 2024. USENIX Association. URL https://www.usenix.org/system/files/usenixsecurity24-dramko.pdf.

Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Peter West, Chandra Bhagavatula, Ronan Le Bras, Jena D. Hwang, Soumya Sanyal, Sean Welleck, Xiang Ren, Allyson Ettinger, Zaid Harchaoui, and Yejin Choi. Faith and fate: Limits of transformers on compositionality. In *Advances in Neural Information Processing Systems*, volume 36. Curran Associates, Inc., 2023. URL https://openreview.net/forum?id=Fkckkr3ya8.

Haeun Eom, Dohee Kim, Sori Lim, Hyungjoon Koo, and Sungjae Hwang. R2I: A relative readability metric for decompiled code. *Proceedings of the ACM on Software Engineering*, 1(FSE):383–405, 2024. doi: 10.1145/3643744. URL https://doi.org/10.1145/3643744.

Joseph A. Fisher. Very long instruction word architectures and the ELI-512. In *Proceedings of the 10th Annual International Symposium on Computer Architecture*, ISCA '83, pages 140–150, New York, NY, USA, 1983. Association for Computing Machinery. doi: 10.1145/800046.801649. URL https://doi.org/10.1145/800046.801649.

John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6 edition, 2019.

Iman Hosseini and Brendan Dolan-Gavitt. Beyond the C: Retargetable decompilation using neural machine translation. *arXiv preprint arXiv:2212.08950*, 2022. doi: 10.48550/arXiv.2212.08950. URL https://arxiv.org/abs/2212.08950.

Intel Corporation. *Intel Itanium Architecture Software Developer's Manual, Volume 3: Instruction Set Reference*. Intel Corporation, October 2002. URL https://www.intel.com/content/dam/www/public/us/en/documents/manuals/itanium-architecture-vol-3-manual.pdf. Revision 2.1.

Intel Corporation. *Intel IXP2400 Network Processor Hardware Reference Manual*. Intel Corporation, November 2003. URL https://www.cs.cmu.edu/~prs/cap/extern/IXP2400/IXP2400HRM.pdf. Revision 007.

Omer Katz, Yuval Olshaker, Yoav Goldberg, and Eran Yahav. Towards neural decompilation. *arXiv preprint arXiv:1905.08325*, 2019. doi: 10.48550/arXiv.1905.08325. URL https://arxiv.org/abs/1905.08325.

Soomin Kim, Markus Faerevaag, Minkyu Jung, SeungIl Jung, DongYeop Oh, JongHyup Lee, and Sang Kil Cha. Testing intermediate representations for binary analysis. In *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering*, ASE 2017, pages 353–364. IEEE Computer Society, 2017. doi: 10.1109/ASE.2017.8115648. URL https://doi.org/10.1109/ASE.2017.8115648.

Jeremy Lacomis, Pengcheng Yin, Edward J. Schwartz, Miltiadis Allamanis, Claire Le Goues, Graham Neubig, and Bogdan Vasilescu. DIRE: A neural approach to decompiled identifier naming. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering*, ASE '19, pages 628–639. IEEE Press, 2019. doi: 10.1109/ASE.2019.00064. URL https://doi.org/10.1109/ASE.2019.00064.

Chris Lattner and Vikram S. Adve. LLVM: A compilation framework for lifelong program analysis & transformation. In *Proceedings of the International Symposium on Code Generation and Optimization*, CGO 2004, pages 75–88. IEEE Computer Society, 2004. doi: 10.1109/CGO.2004.1281665. URL https://doi.org/10.1109/CGO.2004.1281665.

JongHyup Lee, Thanassis Avgerinos, and David Brumley. TIE: Principled reverse engineering of types in binary programs. In *Proceedings of the Network and Distributed System Security Symposium (NDSS 2011)*, San Diego, CA, USA, 2011. Internet Society. doi: 10.1184/R1/6469466.v1. URL https://www.ndss-symposium.org/ndss2011/tie-principled-reverse-engineering-of-types-in-binary-programs/.

Thomas Lengauer and Robert Endre Tarjan. A fast algorithm for finding dominators in a flowgraph. *ACM Transactions on Programming Languages and Systems*, 1(1): 121–141, 1979. doi: 10.1145/357062.357071. URL https://doi.org/10.1145/357062.357071.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tacl_a_00638. URL https://aclanthology.org/2024.tacl-1.9/.

Zhibo Liu and Shuai Wang. How far we have come: Testing decompilation correctness of C decompilers. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA '20, pages 475–487, New York, NY, USA, 2020. Association for Computing Machinery. doi: 10.1145/3395363.3397370. URL https://doi.org/10.1145/3395363.3397370.

Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. Alive2: Bounded translation validation for LLVM. *Proceedings of the ACM on Programming Languages*, 5(PLDI), 2021. doi: 10.1145/3453483.3454030. URL https://doi.org/10.1145/3453483.3454030.

National Security Agency. *Ghidra Decompiler Concepts*. National Security Agency, 2026. URL https://www.ghidradocs.com/12.0.4_PUBLIC/help/topics/DecompilePlugin/DecompilerConcepts.html. Software documentation for Ghidra 12.0.4; accessed 2026-05-09.

Matthew Noonan, Alexey Loginov, and David Cok. Polymorphic type inference for machine code. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2016, pages 27–41, New York, NY, USA, 2016. Association for Computing Machinery. doi: 10.1145/2908080.2908119. URL https://doi.org/10.1145/2908080.2908119.

Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, Philadelphia, Pennsylvania, USA, 2002. Association for Computational Linguistics. doi: 10.3115/1073083.1073135. URL https://aclanthology.org/P02-1040/.

Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. CodeBLEU: A method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297*, 2020. doi: 10.48550/arXiv.2009.10297. URL https://arxiv.org/abs/2009.10297.

Edward J. Schwartz, JongHyup Lee, Maverick Woo, and David Brumley. Native x86 decompilation using semantics-preserving structural analysis and iterative control-flow structuring. In *22nd USENIX Security Symposium (USENIX Security 13)*, pages 353–368, Washington, DC, USA, 2013. USENIX Association. URL https://www.usenix.org/system/files/conference/usenixsecurity13/sec13-paper_schwartz.pdf.

Hanzhuo Tan, Qi Luo, Jing Li, and Yuqun Zhang. LLM4Decompile: Decompiling binary code with large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 3473–3487, Miami, Florida, USA, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.203. URL https://aclanthology.org/2024.emnlp-main.203/.

Texas Instruments. *TMS320C6000 Assembly Language Tools v8.1.x User's Guide*. Texas Instruments, January 2016. URL https://downloads.ti.com/docs/esd/SPRUI03A/Content/SPRUI03A_HTML/assembler_description.html. Literature Number: SPRUI03A, revised January 2016.

Erik F. Tjong Kim Sang and Fien De Meulder. Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition. In *Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003*, pages 142–147. Association for Computational Linguistics, 2003. URL https://aclanthology.org/W03-0419/.

Tree-sitter contributors. Tree-sitter: An incremental parsing system for programming tools, 2026. URL https://tree-sitter.github.io/tree-sitter/. Software release version 0.26.7; accessed 2026-05-14.

Khaled Yakdan, Sebastian Eschweiler, Elmar Gerhards-Padilla, and Matthew Smith. No more gotos: Decompilation using pattern-independent control-flow structuring and semantics-preserving transformations. In *Proceedings of the 22nd Annual Network and Distributed System Security Symposium (NDSS 2015)*. Internet Society, 2015. doi: 10.14722/ndss.2015.23185. URL https://www.ndss-symposium.org/wp-content/uploads/2017/09/11_4_2.pdf.